



American University of Sharjah
Department of Computer Science and Engineering

Traffic Sign Detection and Localization

This Project is submitted to the department of Computer Science and Engineering at the College of Engineering in partial fulfillment of the requirements for the degree of

**Bachelor of Science Degree in
Computer Science**

Presented By

Aadhith Shankarnarayanan (CMP) - b00089801
Koushal Parupudi (CMP) - b00087520
Vibha Bhavikatti (CMP) - g00089208
Raaed Munshi (CMP) - b00089354

Advisor:

Dr. Imran Zualkernan

Dr. Dana Dghaym

4 May 2024

Table of Contents

<i>Table of Contents.....</i>	<i>2</i>
<i>Abstract.....</i>	<i>5</i>
1. Introduction.....	6
1.1 Motivation.....	6
1.2 Project Description.....	7
1.3 Organization of the Report.....	8
2. Problem Statement and Design Objectives.....	10
2.1 Problem Statement and Proposed Solution.....	10
2.2 Design Objectives.....	10
2.3 Design Limitations.....	11
2.3 Design Assumptions.....	11
3. Literature Review.....	12
3.1 YOLO architecture.....	12
3.2 Edge Devices.....	16
3.3 CNN and Variants.....	17
3.4 Single Shot Detectors.....	18
3.5 Geolocation Applications.....	18
3.6 Summary Tables.....	19
3.6.1 YOLO architecture.....	19
3.6.2 Edge Devices.....	22
3.6.3 CNN and Variants.....	23
3.6.4 Single Shot Detectors.....	25
3.6.5 Geolocation Applications.....	26
4. System Specification.....	27
4.1 Functional Requirements:.....	28
4.1.1 Main System Capabilities.....	28
4.1.2 User Interface System Capabilities.....	29
4.2 Nonfunctional requirements.....	30
4.3 Pseudo Requirements	30
4.4 Use Case Diagram.....	31
4.4.1 Traffic Sign Detection Use Case Diagram.....	31
4.4.2 User Interface Use Case Diagram.....	32
5. Technical Approach and Design Alternatives.....	41
5.1 Problem Statement and Proposed Solution.....	42
5.2 Design of the Solution.....	42
5.2.1 Holistic diagram	42

5.2.2	Component Diagram.....	44
5.2.3	Deployment Diagram.....	46
5.2.4	Sequence Diagram.....	47
5.2.5	Activity Diagram.....	49
5.2.6	Main Resources Needed {on the block level}.....	50
5.3	Alternative Designs.....	51
6.	<i>Project Management Plan</i>	54
7.	<i>Implementation</i>	58
7.1	<i>Dataset Analysis</i>	58
7.1.1	Key Findings From the Dataset.....	58
7.2	<i>Software</i>	60
7.2.1	The Deep Learning Model.....	60
7.2.2	Localization of the Detected Traffic Sign.....	61
7.2.3	Optical Flow Algorithm for Deduplication of Traffic Signs.....	62
7.2.4	Front-end - FlutterFlow Application for the User Interface.....	63
7.2.5	Back End.....	63
7.3	<i>Integration</i>	63
8.	<i>Verification, Validation, and Testing Plan and Result Analysis</i>	64
8.1	Metrics.....	64
8.2	Results Analysis.....	67
9.	<i>Project Global, Economic, Societal Impact</i>	72
10.	<i>Standards</i>	74
11.	<i>Conclusion</i>	76
11.1	Summary.....	76
11.2	Future Work.....	76
	<i>References</i>	78

List of Figures

Figure 1. One-Stage Detector YoloV4 [1]	13
Figure 2. Use Case Diagram of Traffic Sign Detection System	33
Figure 3. Use Case Diagram of User Interface System	34
Figure 4. Holistic diagram of the working of the proposed system	43
Figure 5. Detailed System Architecture	44
Figure 6. Component Diagram	45
Figure 7. Deployment Diagram	47
Figure 8. Sequence Diagram of the system	48
Figure 9. Activity Diagram of the system	50
Figure 10. System Hardware Resources	51
Figure 11. List of Tasks, August to November	56
Figure 12. Gantt Chart	57
Figure 13. List of Task, January to May	58
Figure 14. Gantt Chart	59
Figure 15: Distribution of unique images in the dataset	60
Figure 16: Class Distribution Histogram	60
Figure 17: Distance from Camera to Object Detected [41]	62
Figure 18: Haversine Formula [42]	62
Figure 19: Total Execution Time versus Load	69
Figure 20: Predictions of the ROI using optical flow	71
Figure 21: Pinned Location Markers displaying the detected signs on the Flutterflow app	72
Figure 22: Traffic sign information displayed after user presses on marker	73

List of Tables

Table 1. Literature Review of Papers using YOLO architecture	20-23
Table 2. Literature Review of Papers using Edge devices	23-24
Table 3. Literature Review using CNN and its Variants	24-26
Table 4. Literature Review using Single-Shot Detectors	26-27
Table 5. Literature Review of Geolocation Applications	27-28
Table 6: Comparison of Types of Databases	53
Table 7: Comparison of Types of Cloud Database Services	54
Table 8: Comparison of Frameworks	55
Table 9: Class Reduction	61
Table 10: Results of Model Testing	69

Abstract

Road traffic signs are indispensable for ensuring road safety and the efficient flow of vehicles. However, the degradation or damage of these signs can lead to chaos and accidents, necessitating a reliable and automated solution. Manual inspection of traffic signs, particularly assessing their visibility under varying conditions throughout the day, is prone to errors and is labor-intensive when applied on a large scale. This limitation can result in traffic delays and potential hazards, posing a substantial safety concern. To address this critical issue, our project employs deep learning algorithms to accurately detect visible road traffic signs, identify and eliminate duplicate instances of traffic signs across consecutive images, and estimate its geolocation. Furthermore, the predicted traffic sign with the geolocation of each identified traffic sign is stored in a cloud server and displayed to users through the Flutterflow application. The proposed solution offers a marked improvement over manual inspection, providing a faster and more efficient system for the detection, storage, and display of road traffic sign data. By reducing downtime associated with manual processes, the proposed solution contributes to a streamlined approach in managing and monitoring road traffic signs. Upon training and testing the YOLOv3 model on the dataset obtained from the client, iEngineering, the F1-score attained was 95.60%, the mAP_{0.5} score was 96.18%, while achieving an average inference speed of 44.9 ms per image.

1. Introduction

Advancements in object detection and recognition have seen significant improvements and optimizations amidst the rapid evolution of technology, especially in the fields of neural networks and cloud computing. These advancements have played a pivotal role in the field of developing autonomous vehicles and transportation management systems. Traffic signs are an integral part of implementing safety rules for drivers on roadways. These signs provide drivers with crucial information that helps maintain a decree and decrease mortality due to accidents. Hence, it is of utmost importance that these signs are ensured to be accurate and intact. As manual inspection of these signs would be time consuming and labor intensive, it is also prone to errors. It is crucial that the damaged signs are detected at the earliest as it would lead to potential hazards which poses a significant safety issue. This is where the necessity for traffic sign detection systems arises, where the safety of drivers would be ensured by an automated system. The department responsible for roadway management must be informed about the signs that have incurred damage. This would assist them in promptly replacing damaged signs with new ones wherever necessary.

An existing service in the traffic management industry belongs to the company, iENGINEERING PVT LTD. Based in Virginia in the United States of America, the company's objective is to develop high-quality software and E-Business solutions tailored for the transportation engineering industry. One of their existing services is that of the transportation asset management service, which is responsible for extracting roadway assets from images in a Software as a Service (SaaS) format.

1.1 Motivation

Recent advancements have seen the successful implementation of systems specifically designed for traffic sign detection. However, traffic sign detection and localization with the added objective of deduplication of traffic sign annotations and localization of these traffic signs has not been widely researched and implemented as much as the conventional methods. The groundwork for our project draws inspiration from the existing service developed by iENGINEERING PVT LTD. iENGINEERING Private Limited has already developed their automated asset extraction software in which our algorithm will be embedded.

Building upon this foundation, this project aims to extend the capabilities of roadway sign detection and localization, emphasizing on meeting certain requirements specified by the

stakeholder. This extension aligns with the overarching trend in technology towards making intelligent systems more versatile and optimized for its use-case.

1.2 Project Description

In this project, the aim is to demonstrate the detection and localization of traffic signs. Using deep learning techniques, the goal is to create a system that would provide an implementation of the detection and determination of the location of these signs. The system must be able to classify the signs accurately after detection and determine the geolocation of the detected signs, as well as, deduplicate the classifications to ensure unique annotations. The main requirements of this project are:

1. Capturing images of the traffic sign encountered on the roadways
2. Detection of the traffic signs from the images captured on roadways using an appropriate deep learning algorithm
3. Estimating the geolocation of these detected traffic signs
4. Transmission of the sign data and geolocation to a cloud server
5. Displaying the geolocation of the signs on a user interface such as a web application

The main benefits of this project is the additional element of localization along with the detection of traffic signs and ensuring unique annotations. This is helpful as the estimated geolocation of the signs can be used by traffic management service providers who aim to strengthen the road safety conditions. Without manual inspection, an automated system which performs the above would aid in easing the process while also avoiding any potential hazards on the road in a prompt manner.

1.3 Organization of the Report

This report consists of 10 sections which explore further into the essential aspects of this project and provide a detailed outline of the proposed system. The description below outlines the organization of the report, providing an overview of the content discussed in each section.

- Section 1 gives an introduction to the project and the motivation behind it. It explains the current situation regarding the problem and whether there are existing solutions for it. This section also addresses the main requirements and benefits of this project.
- Section 2 consists of the problem statement and the proposed solution to the problem being addressed in the project. It also describes the design objectives, limitations and assumptions of the project.
- Section 3 constitutes the literature review conducted for the project, where there are summaries listed for 35 papers. These papers are works related to the project and this section examines the methods used for similar problems. A conclusion with all the best results from the existing works is summarized as well.
- Section 4 details system specifications where the functional and non-functional requirements are listed. The use case diagrams and the descriptions are also depicted in this section.
- Section 5 describes the technical approach and design alternatives of the project in detail. It consists of a holistic view of the solution, as well as diagrams such as the detailed system architecture diagram, the component diagram, the deployment diagram, sequence diagram and an activity diagram. Additionally, there are alternative designs that are mentioned which are based on the deductions made from the research conducted through the literature review.
- Section 6 illustrates the project management plan through the use of a Gantt chart, where the timeline, deadlines and milestones of the project are shown.
- Section 7 explores the implementation of the project, where the specifics of developing a deep learning model and software implementation, and the integration of different components in the system are discussed.
- Section 8 is composed of the validation, verification, and performance analysis plan. It describes the various metrics used for the evaluation of this project's performance. It also discusses the different testing methods such as the white box testing and user acceptance testing.

- Section 9 discusses the global, economic, and societal impact of the project and the benefits of this project.
- Section 10 provides the conclusion for the report by summarizing and discussing the future work needed to be executed for the project.
- Section 11 provides a conclusion of the project and explores the evolution of this project to optimize it further for asset management.

2. Problem Statement and Design Objectives

2.1 Problem Statement and Proposed Solution

Recent advancements in neural networks and cloud computing have fueled a growing demand for accurate traffic sign detection and localization in various applications. This project aims to demonstrate the effectiveness of such systems in identifying traffic signs in images, pinpointing their geolocation on US roadways, and ensuring the elimination of duplicate instances to maintain sign uniqueness. Our objective is to enhance the existing service developed by iENGINEERING PVT LTD.

The proposed solution of this project is an approach aimed specifically at roadway sign detection and localization. The focus of this project is to develop a neural network model optimized for the use-case of the stakeholder. The project plan initially aims to analyze the different architectures that have been previously employed for this specific problem. The next step includes acquiring pre-trained models such as the YOLO family models and preprocessing the procured dataset. The data preprocessing steps must be applied to the dataset from the client before the testing and validation processes. After designing the architecture to suit our solution as well as the industry standards, this would be deployed through a FlutterFlow application for users to test. After deployment, the data of the detected signs and geolocation obtained, should be transmitted to the cloud for storage. This data can then be viewed through a user interface, where the detected traffic sign and its location must be displayed as location markers on a dynamic Google maps API.

The end goal of this project is to implement an existing algorithm with proposed modifications which can be integrated into an existing transportation asset management service. This technology can also be extended to autonomous vehicles, drones, and public transportation systems. Our project could be used for building a supplementary interface for the existing service for developing a comprehensive asset inventory for the US Department of Transportation and other agencies in the US.

2.2 Design Objectives

The following are our design objectives:

- Recognize all the various classes of traffic signs using an appropriate deep learning model.
- Implementation of a process or algorithm to identify and eliminate duplicate instances of traffic signs, ensuring unique annotations across consecutive images.
- The client must be able to obtain a precise estimate of the signs' geolocation to enhance the existing transportation asset management service.

2.3 Design Limitations

1. The system is not designed for operation during night time.
2. The proposed system will be unable to detect and localize if the vehicle exceeds a particular speed limit set while testing the system.
3. The system is not designed to operate in extreme weather conditions, such as rain, storms, snow, etc.
4. The system is trained on US Roadway Signs, however, testing will be done on UAE Roadway Signs.
5. The system expects a video or the sequence of images given as input should be taken at regular intervals of time.

2.4 Design Assumptions

1. The proposed system can detect and localize a fixed class of signs for which the model has been trained.
2. The traffic signs along the roadways are clearly visible properly and appear without any distortion.
3. The proposed system is able to operate during daylight hours.

3. Literature Review

In this section, we review the main works related to our project and summarize the findings in each work. At the end of this section, we conclude and outline the main differences between our work and the reviewed works.

3.1 YOLO architecture

In recent years, the field of Object Detection has witnessed remarkable advancements, with Deep Learning techniques becoming a cornerstone for researchers. Specifically, the application of these techniques in traffic sign detection has seen successful implementations. Object Detection, involving localization and classification techniques, plays a pivotal role in identifying and precisely dimensioning objects within an image using bounding boxes. Addressing the challenge of traffic sign detection in vehicles, Dewi et al. [1] introduced YoloV4_1 (You Only Look Once) with Spatial Pyramid Pooling (SPP). The architecture, consisting of the backbone, neck, and detection layers (illustrated in Fig 1.), achieved 99.4% accuracy on the Taiwan Traffic Sign Dataset (TWTSD), consisting of 900 images. The use of SPP within a single CNN layer enables the accumulation of local region features at various scales. This approach fosters a more comprehensive learning of multi-scale object features. The model's performance metrics reinforce its efficiency, with a mAP@0.50 score of 99.3%, Intersection over Union (IoU) at 90.09, and an inference time of 0.0862 seconds. Notably, the model's F1-score of 0.98 underscores its high accuracy in traffic sign detection. This integration of deep learning models and algorithms, particularly the utilization of SPP, showcases a significant advancement in addressing the challenges of traffic sign detection, particularly in the context of vehicles.

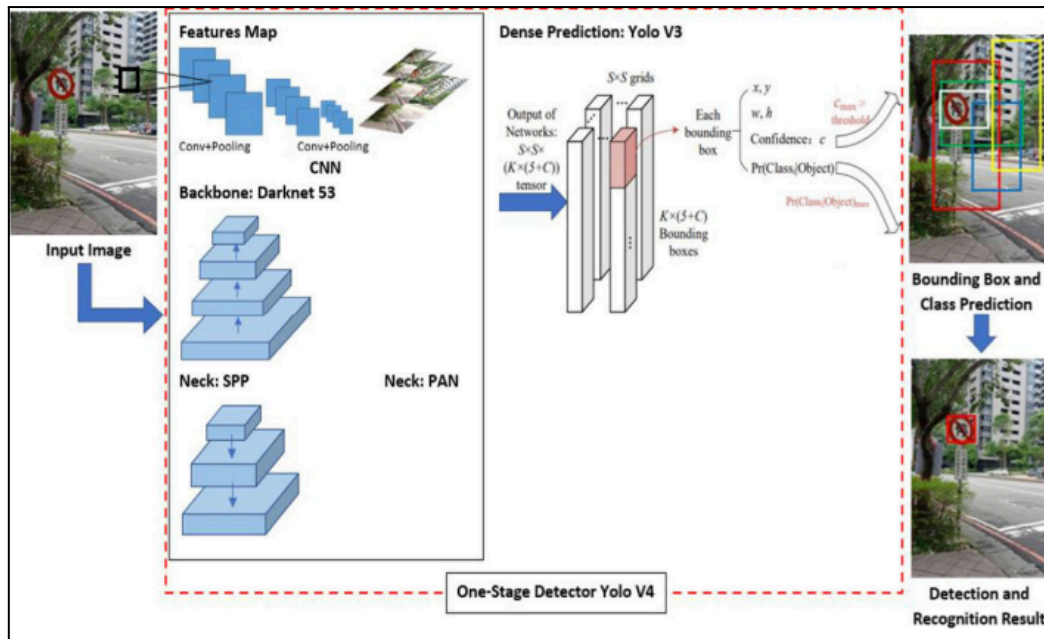


Figure 1. One-Stage Detector Yolo V4 [1]

Employing a strategy similar to [1], Dewi et al. [2] explore the integration of Generative Adversarial Networks (GANs), specifically DCGAN, LSGAN, and WGAN, for data synthesis with the TWTSD. This endeavor aims to enhance the accuracy of traffic sign recognition for both YOLOv3 and YOLOv4 models. The bounding box (BBox) marking tool creates bounding boxes for all signs. As a result, the achieves mAP score of 99.98% and IoU of 90.4 on the YOLOv4 model Moreover, the resulting F1 score of 99.0% further highlights the overall improvement in the accuracy of the YOLOv4 model through the use of synthetic data.

Tang et al. [3] propose a deep learning method for traffic sign recognition based on YOLOv5. Employing the log-average miss rate (LAMR) as the evaluation index, the authors prioritize model performance, with a smaller LAMR indicating superior results. To address imbalances in the Tsinghua-Tencent 100K (TT100K) dataset, the authors introduce the mosaic data enhancement method. This technique involves random scaling, flipping, and adjustments to brightness, saturation, and hue for four images from the training set, subsequently distributing and splicing them into a new composite image. This methodology contributes to achieving a mAP score of 92.1%.

To enhance the efficiency of the YOLOv5s model, Liu et al. [4] proposes a novel approach by substituting the backbone network with MobileNetV2 for feature extraction. Utilizing the

Adam optimizer for network convergence, they conducted a comprehensive analysis on 221 categories from the TT100K dataset, a recognized benchmark for traffic-sign detection and classification. Results demonstrated that the YOLOv5s-MobileNetV2 algorithm outperformed its counterparts, exhibiting an improved detection time of 9 ms compared to the baseline YOLOv5s algorithm, which recorded 10.8 ms. Moreover, the accuracy rate of 91.5% and an mAP@0.5 of 76.5% was achieved. Hu et al. [5] proposed an algorithm that strikes a balance between mean average precision (mAP) at 0.5 and frames per second (FPS), with a slight increase in computational complexity compared to other available models. Using models and algorithms such as Yolov5l, GSconv, Parallel Deformable Convolution Module (PDCM), and Sub-Pixel Convolution Attention Module (SCAM), their model was tested on the TT100K dataset, achieving an mAP@0.5 score of 89.2%. In a parallel effort, Li and Su [6] enhanced the YOLOv5 model by advocating for the replacement of the residual block with two blocks, emphasizing the preservation of feature information. The proposed model aimed to reduce parameters and computation, incorporating a 1x1 convolution at the block's input and a Squeeze and Excitation (SE) block at the end for channel recalibration. The achieved result was an mAP@0.5 score of 76.2%. Zhang et al. [7] introduced a real-time detection method for small traffic signs based on YOLOv3. This method utilized data augmentation, a multi-scale spatial pyramid pooling block, and a bottom-up augmented path to achieve precise localization and classification. Testing on the TT100K dataset yielded notable outcomes, with results indicating 23.8 FPS and an 86% mAP. Jiang et al.'s [8] exploration of the YOLOv5 architecture focused on refining the feature fusion and extraction modules. They incorporated a balanced feature pyramid structure and a global context block, leading to 89.7% mAP@0.5 and 69.3% mAP@0.5:0.95. Furthermore, Xi and Geng [9] conducted tests on YOLOv5s, implementing a reduction in the downsampling layer of the backbone on the TT100K dataset. The results were 96.1% mAP@0.5 and 81.6% mAP@0.5:0.95.

Zhao et al. [10] conducted another study on the TT100k dataset, focusing on addressing the challenges associated with deploying highly complex computational traffic sign detection models on edge devices. To overcome these challenges, they proposed the SEDG-Yolov5 model, an enhanced version based on knowledge distillation and GSConv in the feature fusion layer. This model incorporated the Slicing Aided Hyper Inference method and an inverted residual structure ESGBlock with a fused attention mechanism to resolve issues related to local offline data clouds, high-dimensional information loss, and model complexity. Zhao et al.[10] tested their models on both the TT100K and Swedish traffic sign dataset

(STSD). On TT100K, Yolov5s achieved an $mAP@0.5$ of 91.5%, and SEDG-Yolov5 matched this performance but demonstrated higher frames per second (FPS) on GPU (370.0) and CPU (35.6). On STSD, Yolov5s achieved an $mAP@0.5$ of 93.3%, while SEDG-Yolov5 achieved 92.9% but with a lower number of parameters (2.77M) and higher average precision and recall rates compared to other models. Interestingly, while [9] achieved higher results through a reduction in the downsampling layer, [10] adopted a more complex method with slightly lower results. Building on the YOLOv5s model, another study [11] introduced two traffic sign detection models: YOLOv5-DH and YOLOv5-TDHSa, featuring enhancements. In these models, the last layer of the backbone was replaced with a transformer self-attention module, and the YOLOv5s coupled head was substituted with a decoupled head (DH in both models' names) to enhance detection accuracy and convergence speed. Tested on the TT100k dataset and the CCTSDB dataset, these models achieved an F1 score of 81.1% and 72.0%, respectively, and mAP scores of 83.4% and 69.8%, respectively.

Li et al. [12] proposed a novel YOLO v5 architecture incorporating the C3 Ghost module as a replacement for the YOLO v5 backbone. This module, coupled with a multi-scale feature fusion scheme, aimed to enhance the detection of smaller traffic signs. Utilizing the TT100K dataset, their results demonstrated a commendable $mAP@0.5$ of 91.9%. Building on this work, Zhang et al. [13] introduced the ghost-YOLO, leveraging the same C3 Ghost module and dataset. Their results surpassed [12], showcasing an $mAP@0.5$ of 92.7% and an $mAP@0.5:0.95$ of 73.3%. To optimize accuracy for embedded and low-computing-power mobile devices, Zhao and Danjing [14] presented the G-GhostNet-YOLOx network. This lightweight network replaced the YOLOx's backbone with the G-GhostNet network, aiming to improve feature extraction performance while reducing the model size compared to the original CSPDarknet backbone. Testing on the Chinese traffic dataset yielded results, including 86.7 FPS, 85.3% F1-score, and 75.3% recall.

Song and Suandi [15] adopted a novel approach to enhance the feature extraction layers of YOLOv4-tiny, incorporating dense Spatial Pyramid Pooling (SPP) and fusing input feature layers. Additionally, they utilized k-means clustering to refine neural network traffic sign recognition. Testing their model on the Chinese Traffic Sign Detection Benchmark 2021 (CCTSDB2021) yielded promising results, achieving an F1-score of 87.37%. Avramovic et al. [16] proposed a strategy employing YOLOv3, YOLOv4, and TinyYOLO, reformulating the backbone to utilize up to three Region of Interest (ROI). These ROIs were processed in parallel on multiple GPUs, enhancing the network's focus on specific image regions and

subsequently increasing processing speed. On the DFG dataset, one GPU achieved an FPS of 8, while with three GPUs in parallel, it reached 17. Moreover, the mAP@0.5 for YOLOv3, YOLOv4, and TinyYOLO was reported as 91.87%, 93.44%, and 87.44%, respectively. Gu and Si [17] presented a lightweight real-time traffic sign detection integration framework based on YOLO, aiming to reduce computational overhead and improve detection performance across diverse environments. Striking a balance between detection performance and deployment difficulty, their model achieved results on the German Traffic Sign Detection Benchmark (GTSDB) dataset which included an inference speed of 17.2 ms, 57 FPS, and an mAP of 80.62. Combining the YOLOv5 architecture and Convolutional Neural Network (CNN), Al Khafaji and El Abbadi [18] sought to enhance detection accuracy and processing speed. Training CNN on the German Traffic Sign Recognition Benchmark (GTSRB) dataset and YOLOv5 on the GTSDB dataset, they achieved an accuracy of 94.0% and 99.95%, respectively. Despite this achievement, the system's overall processing speed was limited to 0.031 FPS.

3.2 Edge Devices

With the rapid advancement of technology, the demand for real-time detection has escalated, prompting the utilization of edge devices for efficient data collection. Efforts have been made in achieving real-time inference capabilities in neural networks, as demonstrated by Liu et al. [19]. They proposed an object detector based on the YOLO framework, implemented in real-time on the Nvidia Jetson AGX Xavier edge computing platform. The EdgeYOLO model underwent testing on popular object detection datasets, namely MS COCO2017 and VisDrone2019-DET. Utilizing ELAN Darknet as the EdgeYOLO model's backbone and training it with the stochastic gradient descent (SGD) optimizer, Liu et al. found that their model achieved an accuracy of 50.6% AP@0.50:0.95 and 69.8% AP@0.5 on the MS COCO2017 dataset. The project revealed that smaller, edge-optimized models with fewer parameters exhibited superior performance, especially in terms of small object detection. However, Chang-il et al. [20] identified a gap in the evaluation of model performance in different road environments, prompting their comparison of various models in diverse situations. Constructing their dataset from video data collected on 21 highways in Korea, they assessed Yolov3, Yolov4, and Yolov5s alongside DeepSORT and strongSORT. The study found that Yolov5s with strongSORT achieved the best performance, with an mAP@0.5 of 97.7% and 133 FPS. Moreover, Han and Orukulu [21] presented a traffic sign recognition system based on the NVIDIA Jetson TX1 for driver assistance. Integrating color

and shape-based detection to locate signs and incorporating a pre-trained CNN for classification, the system demonstrated a 96% detection accuracy on the GTSRB dataset. The system processed video at 1.6 FPS.

3.3 CNN and Variants

In contrast to prior methodologies, Tabernik and Skocaj [22] opted for a convolutional neural network (CNN) approach, specifically employing the Mask R-CNN, for traffic sign detection and recognition. The authors conducted a thorough evaluation of their approach on the STSD, establishing a baseline in comparison to related works. To train the model, they utilized the DFG traffic sign dataset. Implementing domain-specific enhancements to the Mask R-CNN, the authors introduced features such as Online Hard Example Mining, where regions of interest are selected based on classification loss value and passed into the learning module. Additionally, they incorporated data augmentation techniques to generate synthetic data, thereby increasing the number of training samples. The achieved metrics include an mAP@0.5% of 95.2%, a recall rate of 96.7%, and a precision of 97.5%.

Megalingam et al. [23] present the Refined Mask R-CNN, introducing a customized dataset based on Indian traffic signs with the aim of reducing traffic accidents in India. In comparison to Fast R-CNN and Mask R-CNN, their results show an average precision of 97.08%, average recall of 96.75%, and an average F-1 score of 96.87%, respectively. Guney and Bayilimis [24] employed the Faster R-CNN architecture, a well-established two-stage approach for object detection. Their model trained in a computer test environment over several hours and iterations. Tested on the German Traffic Sign Dataset (GTSD), it achieved an accuracy of 89.0%. Youssouf [25] implemented Faster R-CNN, training the model on the GTSDb dataset. However, in comparison to YOLOv4, Faster R-CNN performed worse, yielding 6 FPS and 43.3% mAP, while YOLOv4 achieved 35 FPS and 59.9% mAP. Wang et al. [26] delved into traffic sign recognition using Faster R-CNN with Res2Net as the backbone on images of varying sizes in the TT100K dataset. The model exhibited accuracies of 79.8%, 93.9%, and 93.7% for small, medium, and large images, respectively. This highlighted the model's challenge in accurately detecting small images.

Mishra and Goyal [27] implemented a CNN with an Adam optimizer across various datasets, namely GTSRB, GTSDb, BTSC, and TSRD, achieving remarkable accuracy results of 99.8%, 99.8%, 99.8%, and 98.4%, respectively. In a parallel effort, Bouaafia et al. [28]

explored the effectiveness of CNN architecture variants (VGG16, VGG19, AlexNet, and ResNet50) on the GTRSB dataset. Their results demonstrated high accuracy across the variants, with values of 99.3%, 99.5%, 99.7%, and 98.6%, respectively. Additionally, F1-scores for the variants were reported as 93.6%, 93.9%, 95.3%, and 93.7%, respectively. Zhang and Huang [29] conducted a study on the GTRSB dataset, implementing CNN with multi-scale feature fusion, global average pooling, and batch normalization. Their model achieved an inference time of 0.22ms and an accuracy of 99.7%. Exploring a different approach, Abdel-Salam et al. [31] introduced the Real-Time Image Enhanced CNN (RIECNN), incorporating the Image Contrast Enhancement Algorithm and the Multi-Scale Retina Algorithm on images from datasets like GTSRB, BTSD, and rMASTIF. Their novel approach resulted in an average accuracy of 99.8%, 99.2%, and 99.6% for the respective datasets.

3.4 Single Shot Detectors

Ren et al. [32] presented a real-time traffic sign detection system named DS-DesNet, accompanied by a lightweight fusion feature pyramid network (LFFPN), with a primary goal of achieving a balance between speed and accuracy. The study involved testing on three different GPUs—Xavier NX, Nano, Pascal GPU—using the GTSRB dataset. The parameters included 5.76 M, FPS values for Xavier NX, Nano, Pascal GPU at 16, 12, and 61, respectively, alongside an mAP@0.5 score of 81.4%. Similarly, Wu and Liao [33] introduced the Single Shot Multibox Detector with Receptive Field Modification (SSD-RP), incorporating the Receptive Field Modification (RFM) technique to enhance both the receptive field and semantic understanding within predicted feature maps. This modification, particularly beneficial for small traffic signs, contributed to an improved detection performance. The model further integrated a Path Aggregation Network, capturing and interpreting visual information across multiple scales. The model was evaluated on the GTSDb and CCTSDb datasets, yielding an inference time of 26 ms, with mAP scores of 95.4% and 95.9%, respectively.

3.5 Geolocation Applications

Tasyurek [34] introduced a novel method for sign detection utilizing the Exchangeable image file format (EXIF). The approach involved object detection, distance estimation, and systems for handling rotation and projection. The computed distance of the detected object was determined from the photographic point of origin. Subsequently, spatial coordinates on Earth

were established by integrating distance, direction, and GPS data, employing rotation and projection techniques. The study employed YOLO V5 and YOLO V6 for testing on Kayseri Metropolitan Municipality (KMM) data, yielding F1-scores of 95.6% and 94.8%, respectively. In a related effort, Pedersen and Torp [35] focused on enhancing the accuracy of traffic sign location understanding. The authors utilized object size (in pixels), GPS coordinates, position, and heading for their approach. Tests were conducted using RetinaNet on a dataset collected from roads in Denmark, resulting in an AP@0.5 score of 50.8% and AP@0.75 score of 36.6%.

3.6 Summary Tables

3.6.1 YOLO Architecture

Reference Number	Model	Type	Dataset & Performance
[1]	YoloV4_1 with Spatial Pyramid Pooling	Bounding Box	TWTSD dataset mAP@0.5: 99.3% IoU: 90.09 Inference time: 0.0862 s
[2]	Yolo V3 and Yolo V4 with DCGAN, LSGAN, WGAN	Bounding Box	TWTSD dataset mAP Yolo V4: 99.98% IoU: 90.35 F1-Score: 0.99
[3]	YOLOv5 model	Bounding Box	TT100K mAP: 92.1% FPS: 28.85 Inference time: 0.034 s Lamr: 0.1071
[4]	YOLOv5s-MobileNetV2	Bounding Box	TT100K mAP@0.5: 76.5% Param/M: 270.9

[5]	PSG-Yolov5 (Yolov5l + GSconv + PDCM + SCAM)	Bounding Box	TT100K FPS: 85.5 mAP@0.5: 89.2%
[6]	YOLOv5s6	Bounding Box	TT100K mAP@0.5: 76.2%
[7]	MSA_YOLOv3	Bounding Box	TT100K mAP: 86.0% FPS: 23.8
[8]	YOLO v5 with a Balance feature pyramid structure and Attention Module	Bounding Box	TT100K mAP@0.5:89.7% mAP@0.5: 0.95: 69.3%
[9]	YOLOv5s with reduction in the downsampling layer of the backbone	Bounding Box	TT100K mAP@0.5: 96.1% mAP@0.5: 0.95: 81.6%
[10]	SEDG-YOLOv5	Bounding Box	STSD dataset and TT100K mAP@0.5:91.0% mAP@0.5:.95: 71.6% Precision:91.2% Recall:85.8% TT100K: SEDG-Yolov5: mAP@0.5:91.0% mAP@0.5:.95: 71.6% STSD: SEDG-Yolov5: mAP@0.5:92.9% mAP@0.5:.95: 59.4%

[11]	YOLOv5-TDHS A (based on YOLOv5s)	Bounding Box	TT100K F1-Score: 81.1% mAP:83.4% CCTSDb2021: F1-Score: 72.0% mAP: 69.8%
[12]	YoloV5 with Coord-GhostC3 and GhostConv modules	Bounding Box	TT100K mAP@0.5: 91.9% FPS: 360 Params(M): 0.85
[13]	Ghost-YOLO	Bounding Box	TT100K mAP@0.5: 92.7% mAP@0.5:0.95:73.3%
[14]	YOLOx-m by introducing the G-GhostNet lightweight network	Bounding Box	Chinese traffic dataset mAP: 93.4% FPS: 86.7 Recall: 75.3% F1 Score: 85.3%
[15]	TSR-YOLO	Bounding Box	CCTSDb2021 F1 score: 87.4 Recall%: 79.7 Precision%: 96.6 mAP%: 92.7
[16]	YOLOv3, YOLOv4 and TinyYOLO	Bounding Box	FPS (1 GPU in parallel): 8 FPS (3 GPUs in parallel): 17 mAP@0.5: 93.9
[17]	YOLOv4 CSPDarknet53 network	Bounding Box	GTSRB mAP:80.67% Inference time: 17.22ms FPS: 57

[18]	YOLOv5 with CNN	Bounding Box	GTSRB and GTSDDB YOLOv5: Accuracy:94.0% CNN: Accuracy:99.95% FPS: 0.031
------	-----------------	--------------	--

Table 1. Literature Review of Papers using YOLO architecture

3.6.2 Edge Devices

Reference Number	Model	Type	Dataset & Performance
[19]	Edge YOLO	Bounding Box	MS COCO2017 and the UAV object detection dataset VisDrone2019-DET FPS: For 640x384: 50 MS COCO2017 dataset AP@0.5:95:50.6% AP@0.5: 69.8% VisDrone2019-DET dataset AP@0.5:0.95: 26.4% AP@0.5:44.8%
[20]	Yolov3, Yolov4, and Yolov5s	Bounding Box	A vehicle equipped with a physical system collected video data while traveling on 21 highways in Korea, total distance :1246 km FPS: Yolov3(640x640):102 Yolov4(416x416): 123 Yolov4(608x608):70 Yolov5s: 133 Precision: Yolov3:93.5% Yolov4(416x416):87.9% Yolov4(608x608): 88.0% Yolov5s: 95.8% Recall: Yolov3: 94.9% Yolov4(416x416): 88.0% Yolov4(608x608): 89.0%

			Yolov5s: 97.7% mAP@0.5: Yolov3: 95.0% Yolov4(416x416): 77.5% Yolov4(608x608): 83.5% Yolov5s: 97.7% mAP@0.5: 0.95: Yolov3: 81.4% Yolov4(416x416): 71.8% Yolov4(608x608): 77.6% Yolov5s: 78.4%
[21]	DCNN that is modified based on LeNet	Classification	GTSRB FPS:1.6 Accuracy: 96.2%

Table 2. Literature Review of Papers using Edge devices

3.6.3 CNN and Variants

Reference Number	Model	Type	Dataset & Performance
[22]	Mask R-CNN using ResNet-50	Instance Segmentation	DFG Traffic Sign Dataset mAP@0.5: 95.5% Recall: 96.5%
[23]	Refined Mask R-CNN (RM R-CNN)	Instance Segmentation	Custom Indian traffic sign dataset Average Precision:97.08% Average Recall:96.75% Average F1-score:96.87%

[24]	Faster R-CNN	Semantic Segmentation	GTSRB Accuracy: 88.99 Total Loss Rate: 0.220
[25]	Faster R-CNN and YoloV4	Semantic Segmentation and Bounding Box	GTSRB Faster R-CNN mAP: 43.26% YOLOv4: mAP - 59.88%
[26]	Fast RCNN with Res2net	Semantic Segmentation	TTK100K Small images Recall: 87.3 Accuracy: 79.8 % Medium images Recall: 95.8 Accuracy: 93.9 % Large images Recall: 93.2 Accuracy: 93.7% mAP: 87.4%
[27]	CNN with Adam optimizer	Classification	GTSRB Average Accuracy:99.76% GTSDB Average Accuracy:99.81% BTSC Average Accuracy:99.79% TSRD+GTSRB: Average Accuracy:98.37%
[28]	VGG16, VGG19 AlexNet, and ResNet50	Classification	GTSRB VGG16: Average Accuracy:99.33% Average Precision:94.0% Average Recall:94.0% VGG19: Average Accuracy:99.45% Average Precision:93.9% Average Recall:94.3% AlexNet: Average Accuracy:99.74% Average Precision:96.39% Average Recall:94.8%

			ResNet 50: Average Accuracy:98.63% Average Precision:93.4% Average Recall:94.55%
[29]	CNN with multi scale feature fusion, global average pooling and batch normalization	Classification	GTSRB Inference time: 0.22 ms No inception: 98.17% No multiscale feature fusion: 99.07% No global average pooling: 99.32% No Batch normalization: 99.25% Accuracy%: 99.6%
[30]	Haar cascade detector and classification Deep learning CNN model with attention mechanism	Classification	GTSRB ML method: HOG + CLBP + Gabor ELM: 99.10% DL methods: three CNN: 99.70% Precision: 98% Recall: 98% F1 score: 98%
[31]	Real-Time Image Enhanced CNN (RIECNN)	Classification	GTSRB, BTSD, and rMASTIF Average accuracy: GTSRB: 99.75% BTSD: 99.25% rMASTIF: 99.55%

Table 3. Literature Review of Papers using CNN and its variants

3.6.4 Single Shot Detectors

Reference Number	Model	Type	Dataset & Performance
[32]	SSD and lightweight backbone network DS-DetNet	Bounding Box	GTSRB FPS: 12 Params(M): 5.78 mAP:81.35%

[33]	SSD combined with Receptive Field Module (RFM) and PAN	Bounding Box	GTSDDB and CCTSDB GTSDDB Detection time: SSD: 19 ms SSD-RFM+PAN: 26 ms mAP: SSD: 93.2% SSD-RFM+PAN: 95.4% CCTSDB mAP: SSD:93.8% SSD-RFM+PAN:95.9%
------	--	--------------	--

Table 4. Literature Review of Papers using Single Shot Detectors

3.6.5 Geolocation Applications

Reference Number	Model	Type	Dataset & Performance
[34]	Faster R-CNN, YOLO V5, YOLO V6 and transformer-based DETR models using improved supervised learning (ISL), weakly supervised learning (WSL) and fully supervised learning (FSL) techniques	Segmentation and Bounding Box	KMM data F1 score ISL Faster R-CNN: 96.2% ISL YOLO V5:98.5% ISL YOLO V6:96.9% ISL DETR:94.2% FSL Faster R-CNN:90.9% FSL YOLO V5:95.6% FSL YOLO V6 : 94.8% FSL DETR: 92.2%
[35]	RetinaNet provided by Detectron2 framework	Bounding Box	Authors collected data over 2 years in Denmark AP:34.4% AP@0.5:50.8% AP@0.75:36.6%

			APs:16.2% APm:38.7% APl:67.8%
--	--	--	-------------------------------------

Table 5. Literature Review of Geolocation Applications

3.7 Conclusion

In summary, this comprehensive review explores various methodologies in the realm of traffic sign detection and recognition, focusing on diverse deep learning architectures such as YOLO variants, Faster R-CNN, Mask R-CNN, and SSD-RP. The investigation evaluates these models in accurately identifying and classifying traffic signs. Noteworthy enhancements were observed through the incorporation of techniques like GANs for data synthesis, modification of backbone networks, and the introduction of novel modules like Spatial Pyramid Pooling (SPP) and Path Aggregation Network, all contributing to improved precision and efficiency in detection models.

Expanding beyond traditional architectures, the literature review delves into the realm of edge devices. Notably, the adoption of YOLO-based frameworks, exemplified by EdgeYOLO on Nvidia Jetson AGX Xavier, showcases promising results, achieving real-time performance with heightened accuracy, particularly in small object detection scenarios. The review further explores CNN-based approaches, with studies leveraging architectures such as VGG16, VGG19, AlexNet, and ResNet50.

Moreover, the review addresses geolocation aspects, highlighting approaches that involve object detection from EXIF data and incorporate distance estimation, rotation, and projection systems. These techniques aim to determine the spatial coordinates of identified traffic signs on Earth, aligning with our objective for precise location information of traffic signs in intelligent transportation systems.

With these insights, the project will commence implementation, focusing initially on YOLO v4 and YOLO v5. The subsequent stages involve fine-tuning and configuring the dataset provided by iENGINEERING, using a similar methodology to the one presented in [20]. Finally, after refining the model with the dataset, the project will progress into the development of an end-to-end solution.

4. System Specification

The design objectives of this project are as follows:

- Recognize all the various classes of traffic signs using an appropriate deep learning model, YOLOv3, for object detection and classification.
- Implementation of a process or algorithm to identify and eliminate duplicate instances of traffic signs, ensuring unique annotations across consecutive images.
- Obtain a precise estimate of the signs' geolocation to enhance the existing transportation asset management service.

4.1 Functional Requirements

We have categorized the functional requirements into two perspectives: the first aspect centered around the core capabilities of the main system in the project that we are developing, while the second focuses on the User Interface aspect.

Our End Goal Objectives are defined as follows:

1. **Accurately Identify and Classify Traffic Signs:** Develop an algorithmic solution that effectively identifies and classifies traffic signs using labeled crops and adhering to MUTCD guidelines.
2. **Eliminate Duplicate Traffic Sign Instances:** Implement a process to remove duplicate instances of traffic signs, ensuring only unique annotations persist, preventing repetition across consecutive images.
3. **Determine Precise Geolocations:** Develop an algorithm to determine the precise geolocation of each annotated traffic sign using available 2D image data and associated geocoordinates.

4.1.1 Main System Capabilities

FR1. Capture and upload images of roadways

Users can capture images using their mobile phone or upload images of roadways for the deep learning model to recognize traffic signs

FR2. Acquire the coordinates of the camera

The system must acquire the geolocation (latitude and longitude) of the camera used when the image was captured

FR3. Image Preprocessing

The system will apply appropriate image preprocessing techniques including but not limited to image resizing, color correction, and noise reduction would be performed.

FR4. Traffic Sign Detection

The system must utilize an optimal object detection model to accurately detect the bounding boxes of the traffic signs in the roadway images.

FR5. Traffic Sign Recognition

The deep learning model in the system must classify the traffic signs detected into their respective classes: Stop, Pedestrian crossing or No U-turn sign for instance, efficiently. Furthermore, multiple frames of the same road sign would be counted as a unique road sign

FR6. Calculate the Geolocation of the Traffic signs

The system must accurately calculate the geolocation of the traffic signs based on the acquired geolocation of the smartphone and the distance of the sign obtained from the deep learning model.

FR7. Deduplicate Traffic Sign Annotations by using Optical Flow

Predict whether the sign will appear in the next image using optical flow, once it is not appeared, a sequence will be created for the respective sign using all the previous images in the sequence.

FR8. Transmit the data of the Traffic signs and their Geolocation

The system must send the data of the detected traffic signs and their geolocation via the model and geolocation algorithm to the Cloud database server.

FR9. Store the Traffic sign data

The cloud database server in the system receives the data from the model and the algorithms and stores this data which can then be accessed by the User Interface system.

4.1.2 User Interface System Capabilities

FR10. Register

The system must allow users and admins to register into the system by providing required information, such as their email address and password.

FR11. Login

The system must allow users and admins to log in using their email address and the password they registered with and these details will be authenticated.

FR12. Sign Out

The system must allow users and admins who are logged into the system to exit the session by signing out.

FR13. View Data of Traffic Sign

The system must allow users and admins, who are logged into the system, to view traffic sign data and their respective geolocation via a dynamic map API. The map should display location markers for signs, which are clickable to view the metadata.

FR14. Upload Images

The user can upload an image or a batch of images to the traffic sign processing pipeline.

4.2 Nonfunctional requirements

The nonfunctional requirements for the proposed system can be divided into Performance, Quality, Safety, and Interface. The following are the nonfunctional requirements in each category:

NFR1. Performance:

- a. The system must accurately detect road signs with a minimum mAP@0.5 90%, recall of 93%, and F1-Score of 93%.
- b. The system must predict traffic sign classes in less than 2000 milliseconds

NFR2. Quality

The application must be both user friendly and intuitive and show traffic signs using location markers on a map.

NFR3. Safety

The system should implement preventive measures to avoid unintended states, such as freezing or crashing the application, that could cause delays or any other adverse consequences.

NFR4. Security

- a. The system is required to safeguard stored data and code, implementing measures to prevent unauthorized access and data breaches while ensuring the implementation of appropriate authentication measures.
- b. The system should undergo periodic security audits to protect against malicious code injection and exploitation.

NFR5. Ethical Considerations

- a. The system must prioritize user privacy ensuring that the data captured is anonymized and does not include personally identifiable information (such as people, number plates of vehicles, etc).
- b. The system must comply with the UAE and international regulations related to data privacy, security, and the use of computer vision technologies in public spaces.

4.3 Pseudo Requirements

The main constraint implied by the company iEnginnering according to their requirements, is to integrate the system they have worked on with the development being made through our project.

4.4 Use Case Diagram

4.4.1 Traffic Sign Detection Use Case Diagram

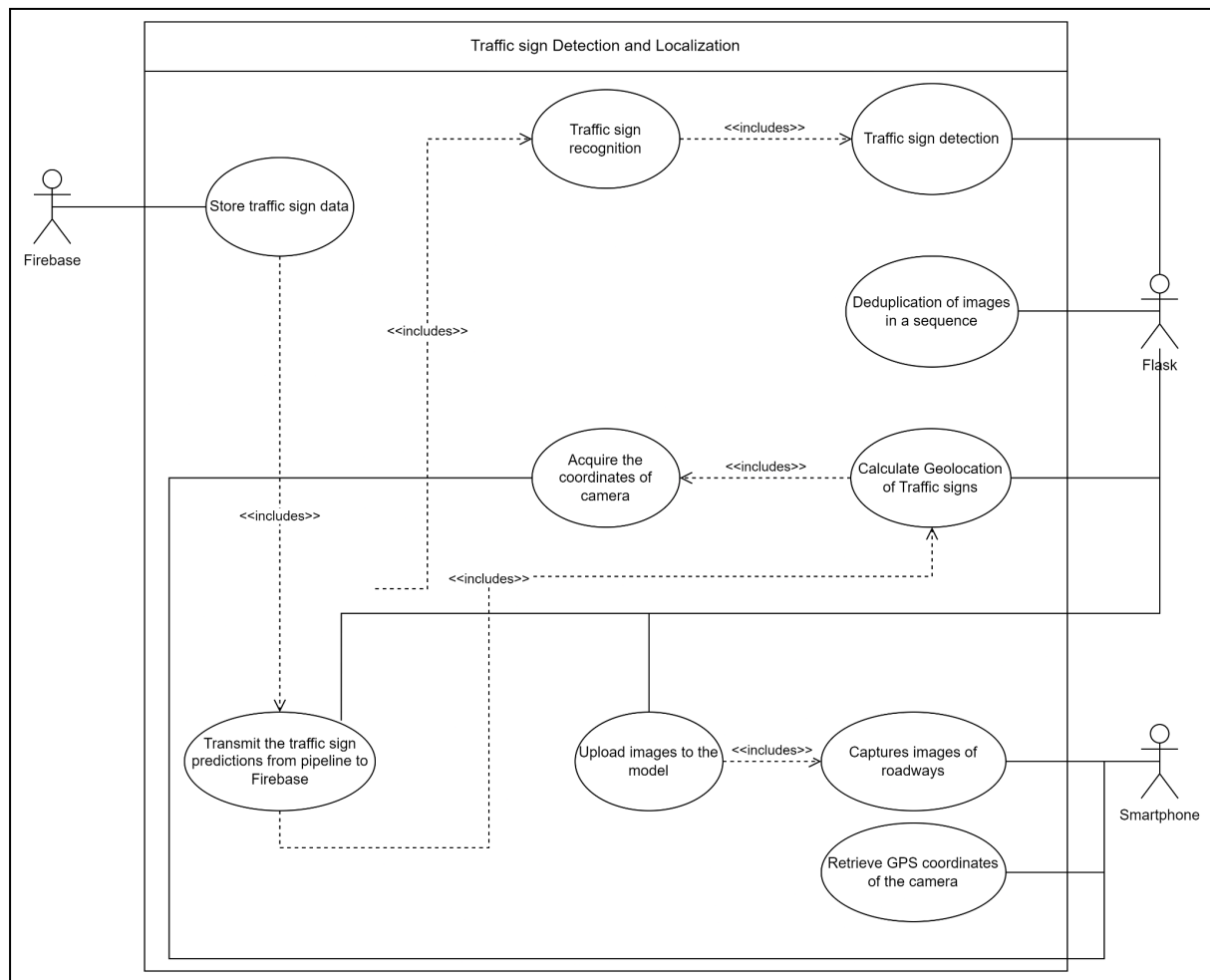


Figure 2. Use Case Diagram of Traffic Sign Detection System

As shown in Figure 2, the Use Case diagram of the Traffic Sign Detection System involves 3 key actors: Smartphone, Flask, Firebase. The camera transmits images to the deep learning model hosted by Flask web framework. Simultaneously, the GPS module in the smartphone captures the current precise geolocation of the user, which is then tagged to the image sent to the model. Once the images of the roadways are streamed to the model via Flask, the model engages in preprocessing these images and performs traffic sign detection and recognition. Utilizing the geotagged images, we calculate the geolocation of the detected traffic signs. Finally, the predicted traffic sign classes and their respective geolocations are transmitted, via Flask, to Firebase, where the data will be stored and retrieved by the users.

4.4.2 User Interface Use Case Diagram

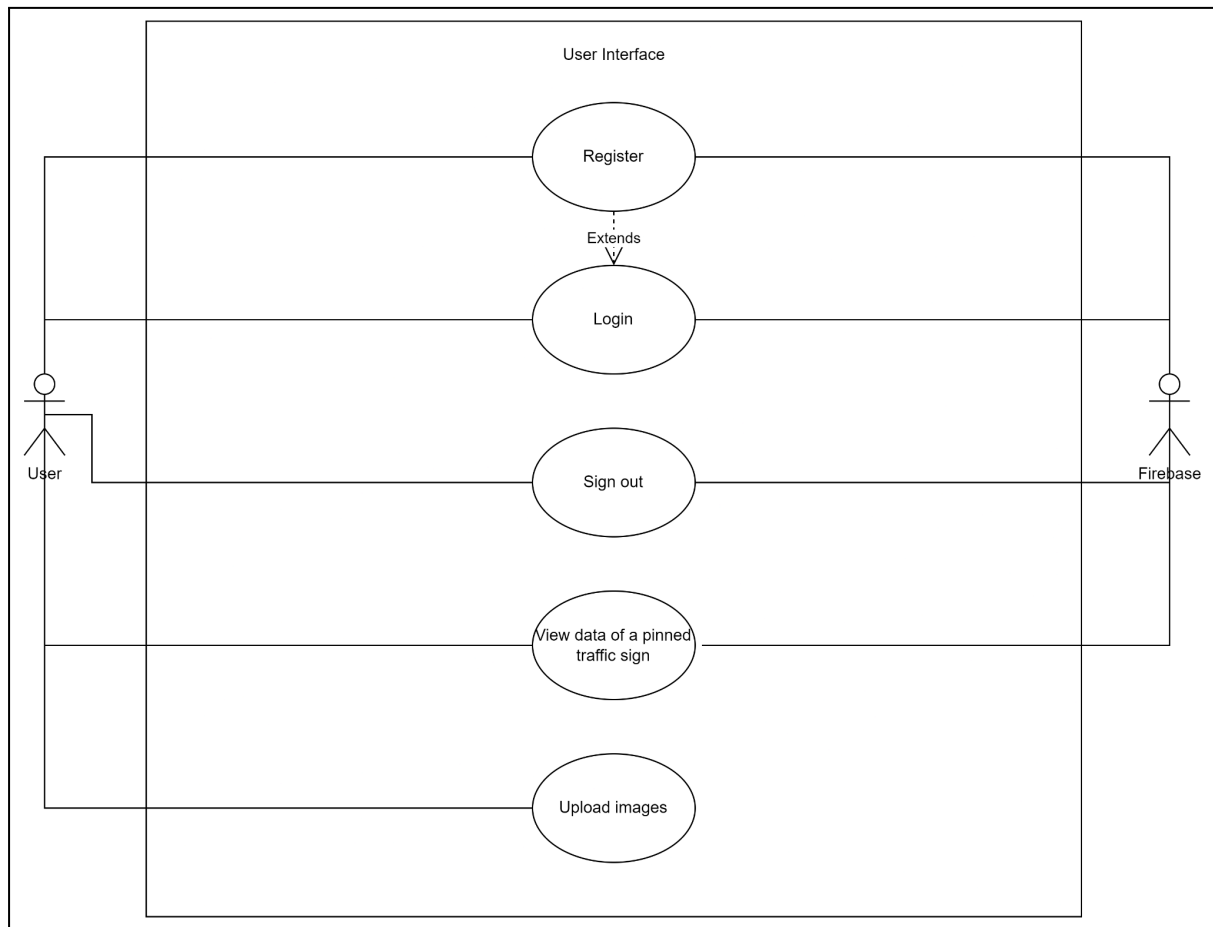


Figure 3. Use Case Diagram of User Interface System

As shown in Figure 3, the User Interface use case diagram involves 2 primary actors: User and Firebase. After establishing a connection with Firebase, a user can register as a standard user using their email address and password on the server, proceeding to login after registration. Once the user's credentials are authenticated, users gain access to view data concerning all detected traffic signs pinned onto a map. Clicking on a specific pin reveals detailed information about the traffic sign. The user can also upload images on the application which are transmitted to the traffic sign processing pipeline to acquire the predictions and geolocation and placed as markers on the map.

4.4.3 Use Case Descriptions

Use case: Store traffic sign data
ID: cloud _1
Brief Description: The backend uploads the extracted data to Firebase.
Primary actors: Firebase
Secondary actors: N/A
Preconditions: Firebase is set up and configured. The traffic sign detection and recognition model is already trained.
Main flow: 1. include(flask_1) 2. The extracted information of the traffic signs is sent from the backend to Firebase
Postconditions: Firebase is updated with the geolocation and the classified traffic signs detected on roadways.

Use case: Traffic sign recognition
ID: flask_1
Brief Description: The deep learning model performs traffic sign recognition for each traffic sign in the given image.
Primary actors: Flask
Secondary actors: N/A
Preconditions: Deep learning model installed & pre-trained
Main flow: 1. include(flask_2) 2. Each traffic sign within the bounding boxes is recognized by the classification algorithm
Postconditions: Traffic signs are classified in the image and are prepared to be sent to Firebase.

Use case: Traffic sign detection
ID: flask_2
Brief Description: The object detection model places bounding boxes to detect traffic signs in a given image.
Primary actors: Flask
Secondary actors: N/A
Preconditions: Deep learning model installed & pre-trained
Main flow: 1. Image is run through a deep-learning model transmitted by Flask to detect individual traffic signs 2. If there exists multiple frames of the same traffic sign, count the traffic sign only once 3. Bounding boxes for the traffic signs are placed 4. include(flask_1)
Postconditions: The bounding boxes are placed on traffic signs in the frames which would be then classified into their respective classes.

Use case: Calculate Geolocation of traffic signs
ID: flask_3
Brief Description: The traffic sign processing pipeline will be able to calculate the geolocation of traffic signs.
Primary actors: Flask
Secondary actors: N/A
Preconditions: The traffic sign detection and recognition algorithm predicted the traffic sign class
Main flow: 1. include(GPS_1) 2. The pipeline calculates geolocation of traffic signs by approximating the distance between the traffic signs and the camera.
Postconditions: The geolocation of traffic signs can be stored in Firebase

Use case: Retrieve GPS coordinates of the camera
ID: cam_2
Brief Description: The pipeline retrieves the GPS coordinates of the camera.
Primary actors: Camera
Secondary actors: N/A
Preconditions: The images are geotagged
Main flow: 1. The coordinates of the camera are sent to the pipeline to estimate the distance of the traffic signs from the camera.
Postconditions: N/A

Use case: Deduplication of images in a sequence
ID: flask_4
Brief Description: The traffic sign processing pipeline performs optical flow to deduplicate the classifications to ensure only unique annotations exist
Primary actors: Flask
Secondary actors: N/A
Preconditions: The optical flow algorithm is set up to deduplicate after the object detection model predicts the traffic sign in the image
Main flow: 1. The optical flow algorithm checks whether the subsequent image contains the same class of traffic sign. 2. If it does, it predicts the ROI points of the next image that the traffic sign will appear in. 3. If the prediction aligns with the actual points, the image will be considered a part of the sequence.
Postconditions: The images are deduplicated and only one image is used to calculate the geolocation.

Use case: Upload images to the model
ID: flask_5
Brief Description: The camera uploads images to the deep learning model via Flask web framework.
Primary actors: Flask
Secondary actors: N/A
Preconditions: The user has taken images of roadways to send to the pipeline and has a registered account on the FlutterFlow application
Main flow: 1. include cam_2 2. Upload images from the camera to the pipeline via Flask
Postconditions: N/A

Use case: Capture images of roadways
ID: cam_1
Brief Description: The user uses a smartphone camera to capture images of roadways
Primary actors: Smartphone
Secondary actors: N/A
Preconditions: N/A
Main flow: 1. User uses a camera app on the smartphone to capture images of the roadways
Postconditions: The images can be uploaded to the model for detection, deduplication and estimating geolocation

Use case: Transmit traffic sign predictions from pipeline to Firebase
ID: flask_6
Brief Description: The model transmit the traffic sign data to Firebase via Flask framework
Primary actors: Flask
Secondary actors: N/A
Preconditions: The traffic sign recognition algorithm has run and completed classification
Main flow: 1. After the traffic sign processing pipeline predicts the sign class, deduplicates the classifications and estimates geolocation, the data is sent to Firebase via Flask
Postconditions: The traffic sign data is transmitted to Firebase where it can be stored.

Use case: Register
ID: User_1
Brief Description: The user registers as a user of the user interface
Primary actors: User
Secondary actors: Firebase
Preconditions: Firebase is set up and configured
Main flow: 1. The user enters their information in the required fields on the user interface 2. Firebase checks if the credentials are unique and valid to be able to register the new user 3. The user account data is added to Firebase
Alternate flow: The credentials entered are invalid: 1. User interface (UI) alerts the user that the provided credentials are invalid 2. The UI asks the employee to re-enter their credentials again 3. extends(Login)
Postconditions: The user is registered on Firebase and can login

Use case: Login
ID: User_2
Brief Description: The user can login with their account on the user interface to allow them to upload images
Primary actors: User
Secondary actors: Firebase
Preconditions: User is already registered, Firebase is set up and configured
Main flow: 1. The user enters their credentials on the user interface. 2. The provided credentials are checked for a match in Firebase 3. Firebase authenticates the user 4. The user is alerted on the user interface that the login was successful
Alternate flow: The credentials entered are invalid: 1. System alerts the employee that the provided credentials are invalid 2. The system asks the employee to re-enter their credentials again
Postconditions: The user can now view the traffic sign data of any pinned traffic sign locations on the UI

Use case: Sign Out
ID: User_3
Brief Description: The user can sign out of their account from the Firebase.
Primary actors: User
Secondary actors: Firebase
Preconditions: User already has an account, and is logged in, Cloud server is set up and configured
Main flow: 1. The user presses/clicks on a Sign Out button on the user interface 2. The user is signed out of their account 3. The user is redirected to a Login page on the user interface
Postconditions: User is signed out application

Use case: View data of pinned traffic sign
ID: User_4
Brief Description: The user can view traffic sign data to the cloud server.
Primary actors: User
Secondary actors: Firebase
Preconditions: User already has an account, and is logged in, and Firebase has data to show on the map
Main flow: 1. The user presses/clicks on a location marker on the user interface 2. The user is presented with traffic sign data along with geolocation on a popup window 3. The user can exit the popup window
Postconditions: N/A

Use case: Upload images
ID: User_5
Brief Description: The user can upload an image or a batch of images to the traffic sign processing pipeline.
Primary actors: User
Secondary actors: N/A
Preconditions: The user has taken images of roadways to send to the pipeline and has a registered account on the FlutterFlow application
Main flow: 1. include cam_2 2. Upload images from the camera to the pipeline through the application
Postconditions: N/A

5. Technical Approach and Design Alternatives

5.1 Problem Statement and Proposed Solution

Recent advancements in neural networks and cloud computing have fueled a growing demand for accurate traffic sign detection and localization in various applications. This project aims to demonstrate the effectiveness of such systems in identifying traffic signs in images, pinpointing their geolocation on US roadways, and ensuring the elimination of duplicate instances to maintain sign uniqueness. Our objective is to enhance the existing service developed by iENGINEERING PVT LTD.

The proposed solution of this project is an approach aimed specifically at roadway sign detection and localization. The focus of this project is to develop a neural network model optimized for the use-case of the stakeholder. The project plan initially aims to analyze the different architectures that have been previously employed for this specific problem. The next step includes acquiring pre-trained models such as the YOLO family models and preprocessing the procured dataset. The data preprocessing steps must be applied to the dataset from the client before the testing and validation processes. After designing the architecture to suit our solution as well as the industry standards, this would be deployed

through a FlutterFlow application for users to test. After deployment, the data of the detected signs and geolocation obtained, should be transmitted to the cloud for storage. This data can then be viewed through a user interface, where the detected traffic sign and its location must be displayed as location markers on a dynamic Google maps API.

The end goal of this project is to implement an existing algorithm with proposed modifications which can be integrated into an existing transportation asset management service. This technology can also be extended to autonomous vehicles, drones, and public transportation systems. Our project could be used for building a supplementary interface for the existing service for developing a comprehensive asset inventory for the US Department of Transportation and other agencies in the US.

5.2 Design of the Solution

5.2.1 Holistic Diagram

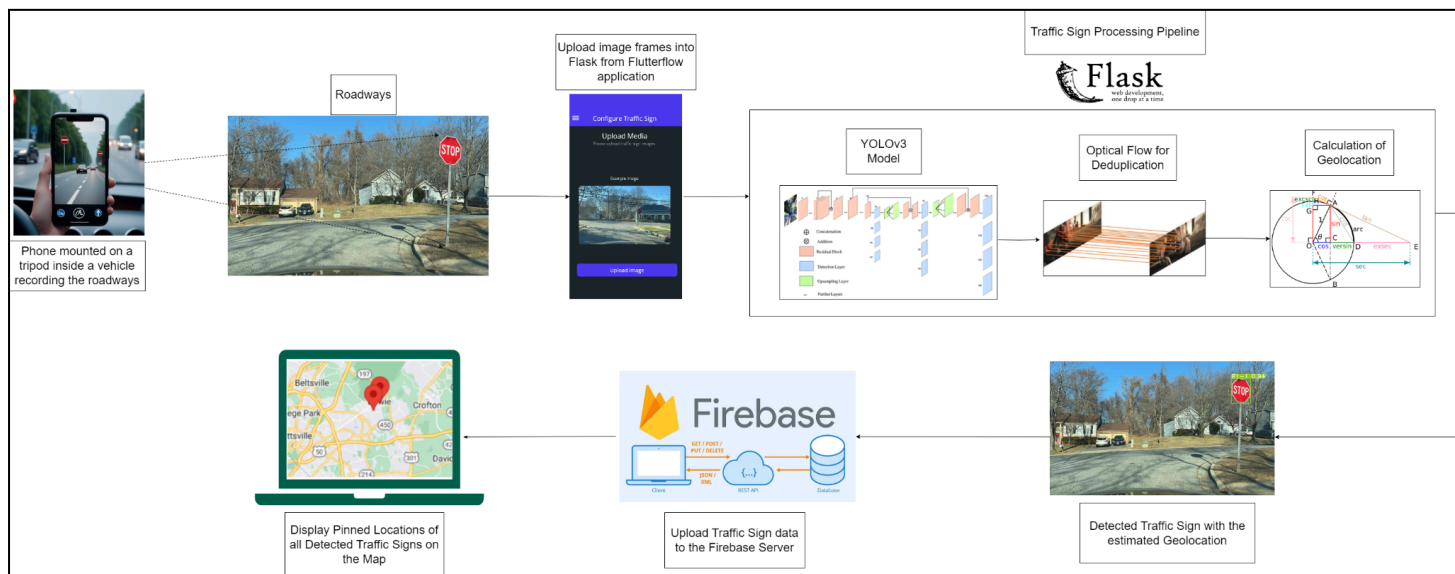


Figure 4. Holistic diagram of the working of the proposed system

According to the system specifications given by iEngineering Private Limited, the system comprises of the components as shown in Figure 4:

1. **Camera:** The smartphone camera mounted on a vehicle will take the images/video of the traffic signs on the road.
2. **Flask:** This part refers to the Flask framework which will receive the captured images. This is where the detection of the traffic signs in the captured images are

performed by the YOLOv3 model, along with the deduplication of signs and estimation of the geolocation of the detected signs. The predicted images consist of the bounding box on the signs.

3. **Cloud-based Storage:** The traffic sign data (predicted images and geolocation) from Flask is transmitted to the Firebase cloud server. This storage option assists in storing and delivering the data from Flask to the Firebase Firestore and Storage, and sends to other systems when requested.
4. **End-user application:** The detected signs would be displayed on the Flutterflow application made for users to view the data.

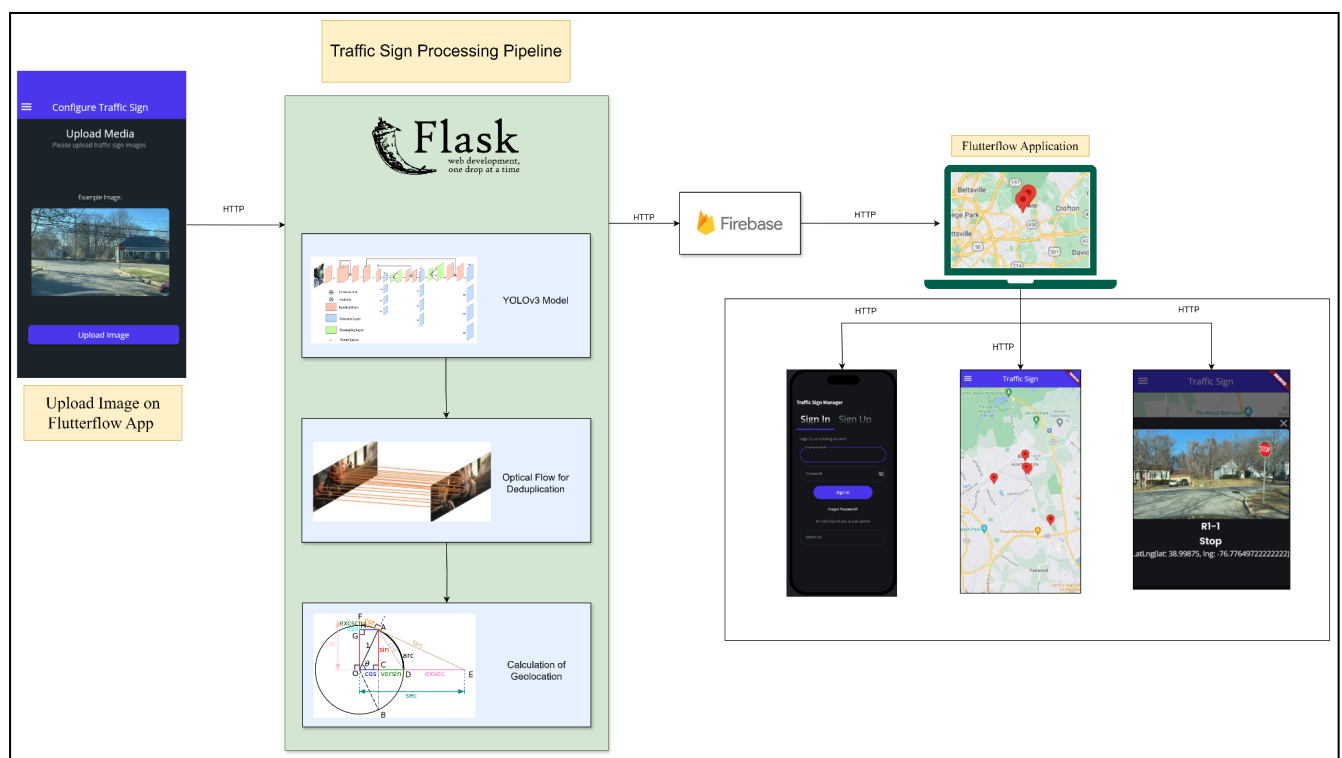


Figure 5. Detailed System Architecture

The holistic diagram and the detailed system architecture of the system, as shown in Figure 5., implements our proposed solution to perform traffic sign detection and localization. Firstly, the smartphone camera is responsible for capturing the images of the traffic signs it encounters. It essentially captures the images and this is used to detect the traffic signs in the next part of the system. This is then sent to Flask, where the images are then given as the input to the deep learning model, where the detection of the signs is performed. The deduplication of frames into unique annotations is performed and subsequently the geolocation of the detected images are also calculated using a custom made algorithm.

The inferences made and geolocation from these are then transmitted to the Firebase cloud storage server. This data can be retrieved anytime from the cloud as it is stored in the database. Any end user can access the detected signs and geolocation through the Flutterflow application. The application can request access to the data on the cloud at any time as per the user's wish.

5.2.2 Component Diagram

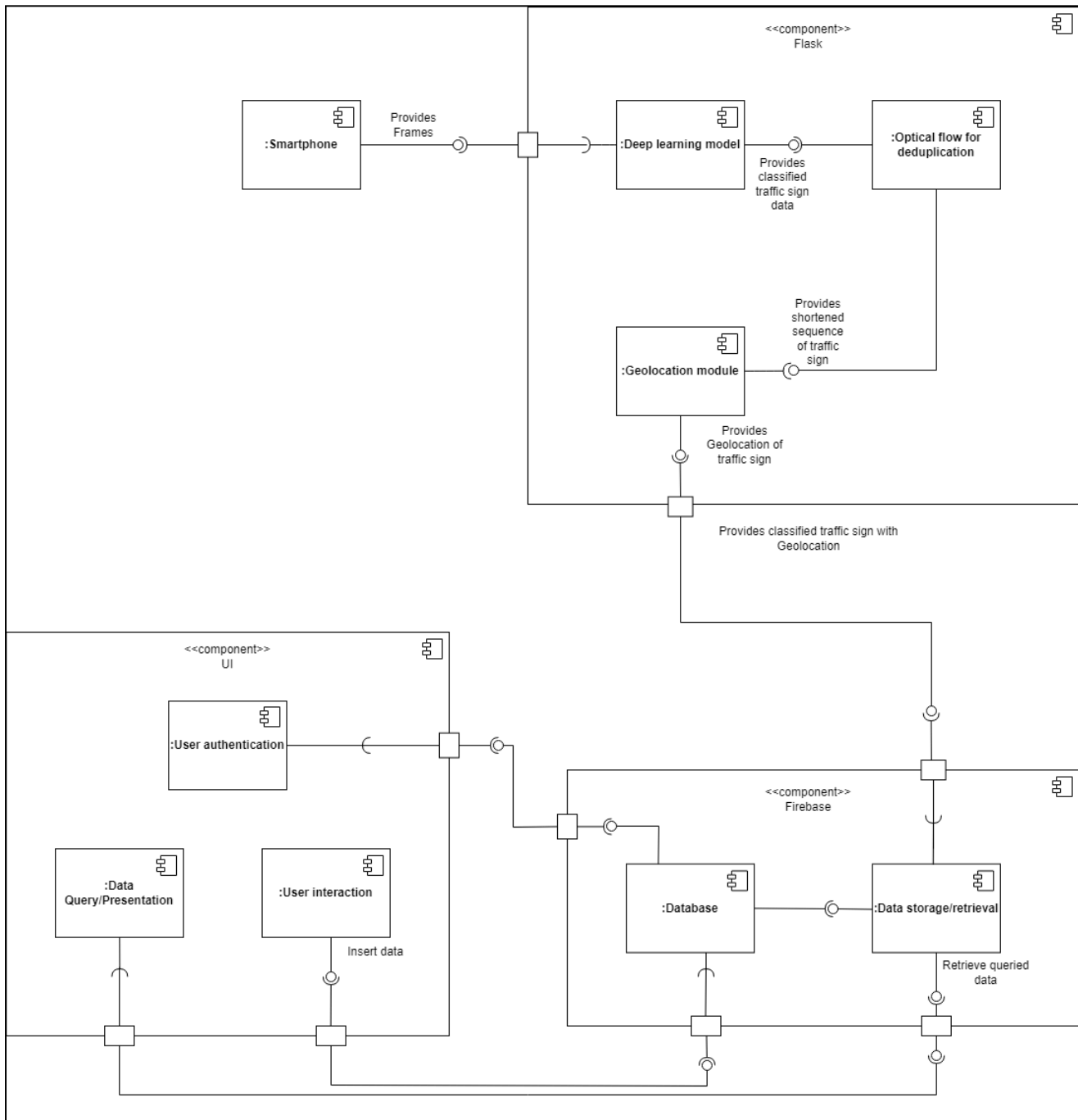


Figure 6. Component Diagram

The component diagram as seen in Figure 6, illustrates the components, their relationships and interactions within the traffic sign detection and localization system.

The system consists of the following components:

1. Flask: This component is responsible for running the deep learning model to detect the traffic signs and classify them, and running optical flow for the deduplication of the traffic sign classes, and running the frames through the geolocation algorithm .
 - a. The deep learning module is a neural network that detects traffic signs and classifies them.
 - b. The optical flow algorithm removes duplicates of traffic signs and makes a shortened sequence of images.
2. Smartphone module: This component captures the images of the roadways
3. UI: This component provides the user interface for the system
 - a. The user authentication module authenticates the user's credentials
 - b. The data query/presentation module is responsible for displaying the traffic sign data the user chooses
 - c. The admin interaction module allows the administrator to modify and delete users and traffic sign data
4. Firebase: This component stores traffic sign data and retrieves the data to display on the user interface

5.2.3 Deployment Diagram

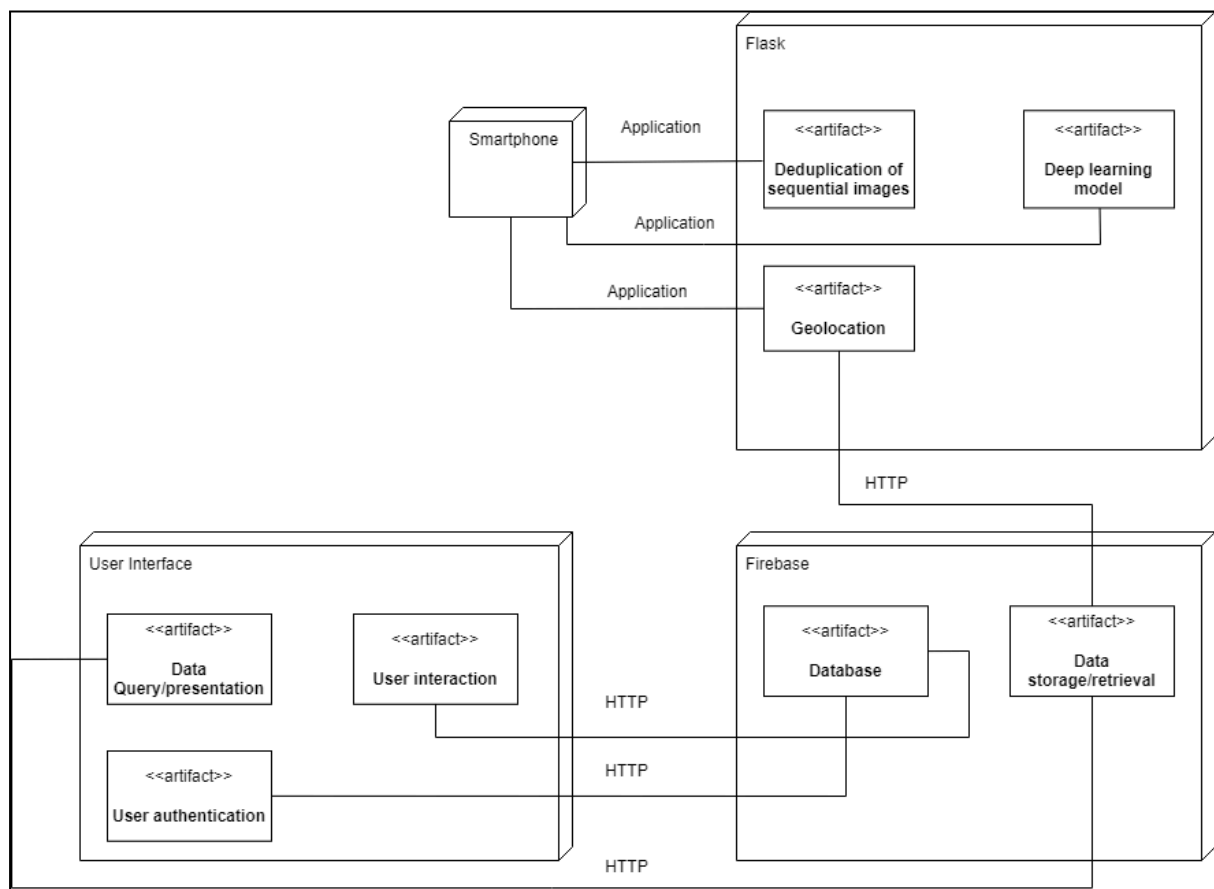


Figure 7. Deployment Diagram

The deployment diagram as seen in Figure 7, illustrates how data is transmitted from component to component. The HTTP protocol transmits classified traffic sign data from Flask to Firebase. The application interface allows the users to use the camera to upload the captured image containing the traffic sign to Flask. This interface is specifically designed for data transfer between camera and Flask while remaining cost-effective. Finally, Flask is connected to Firebase through the HTTP protocol.

The sequence diagram shown in Figure 8 illustrates the sequence of steps taken for the implementation of the system and its functionalities. Users begin by capturing images or videos of the roadways to detect the traffic signs present. As a parallel process, the in-built GPS receiver present in smartphones captures the location data and tags it with the media. The captured data is uploaded via the FlutterFlow application and transmitted to the backend Flask server, where the main processing of data takes place. In the Flask server, the EXIF data is extracted and preprocessed for compatibility with the YOLOv3 model, which facilitates the detection and prediction of the traffic signs. These predicted images undergo our Optical Flow algorithm to deduplicate traffic signs that were detected multiple times. At this stage, the data consists of unique detected traffic signs per geolocation. Precise geolocation determination is performed, and the resulting data, comprising unique detected traffic signs and their geolocations, is uploaded to the Firebase database. Finally, any end user can access the FlutterFlow application to view the pinned locations of detected traffic signs on a map and access additional data regarding the traffic sign upon clicking a location marker.

5.2.5 Activity Diagram

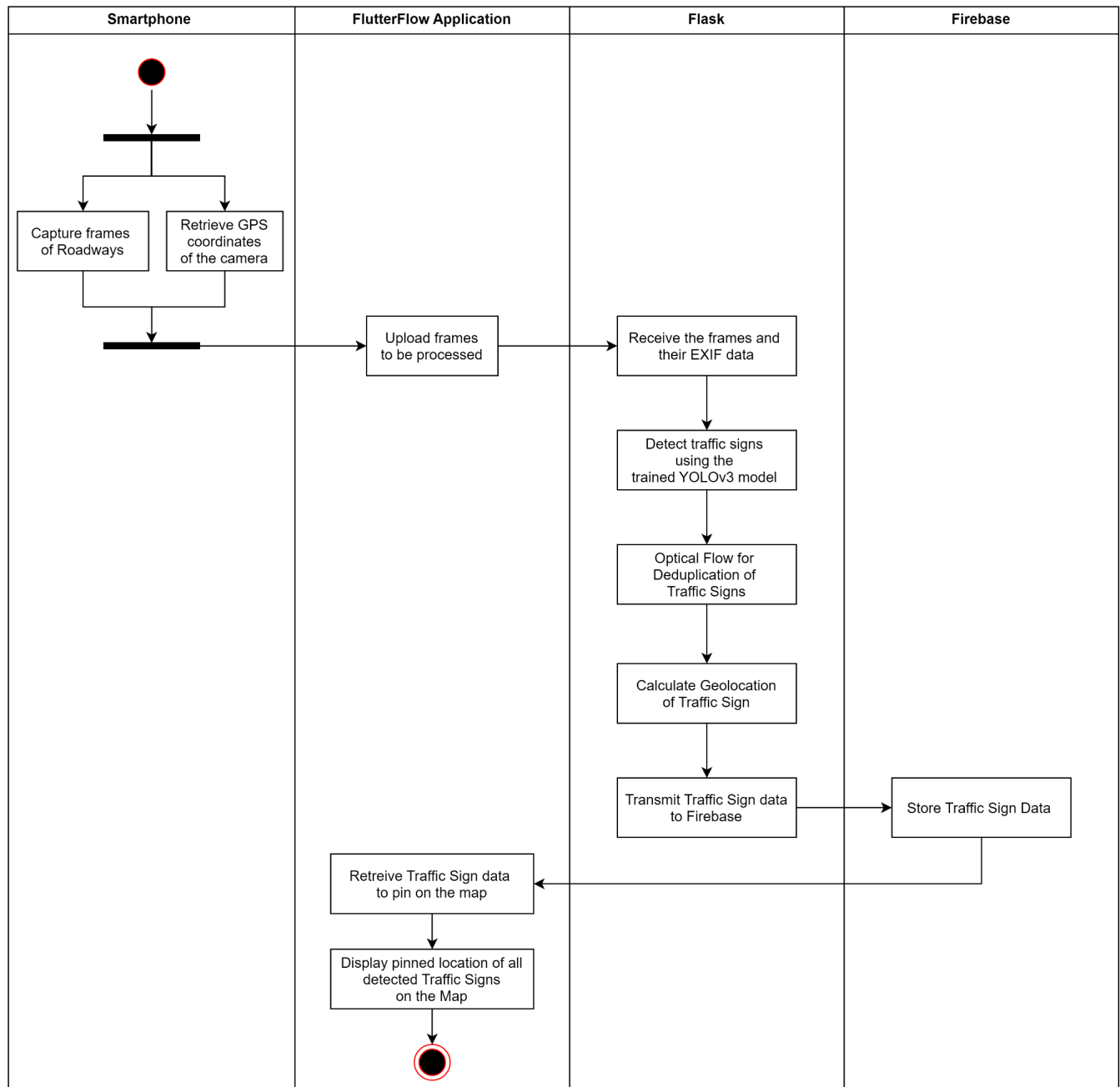


Figure 9. Activity Diagram of the system

The activity diagram shown in Figure 9 illustrates the series of actions and the control flow in a system, which also consists of swimlanes to depict the activities carried out by the different actors. In the initial state, the smartphone camera is activated initially which is responsible for capturing images of the roadways to detect the traffic signs. Another process that happens in parallel is forked with this process, which is the retrieval of the geolocation of the smartphone using the in-built GPS receiver present in smartphones. These processes occur simultaneously and the location data is tagged to the media. The images are then uploaded to be processed

through the FlutterFlow application to the Flask server. In the flask server, the EXIF data contained by the images are extracted to obtain the geolocation when the image was captured. The images are then pre-processed and fed into the trained YOLOv3 model to get the prediction of traffic signs. The images that detected traffic signs are passed through our Optical Flow algorithm to filter out duplicate detections of the same traffic signs at the same geolocation. The resulting set of uniquely predicted traffic sign images undergo calculation of the precise geolocation of the detected signs. The traffic sign data along with the geolocation is then uploaded to Firebase, where the data is stored. Lastly, any end user can view the detected sign data and its location on the FlutterFlow application where the data would be pinned on a map.

5.2.6 Main Resources Needed {on the block level}

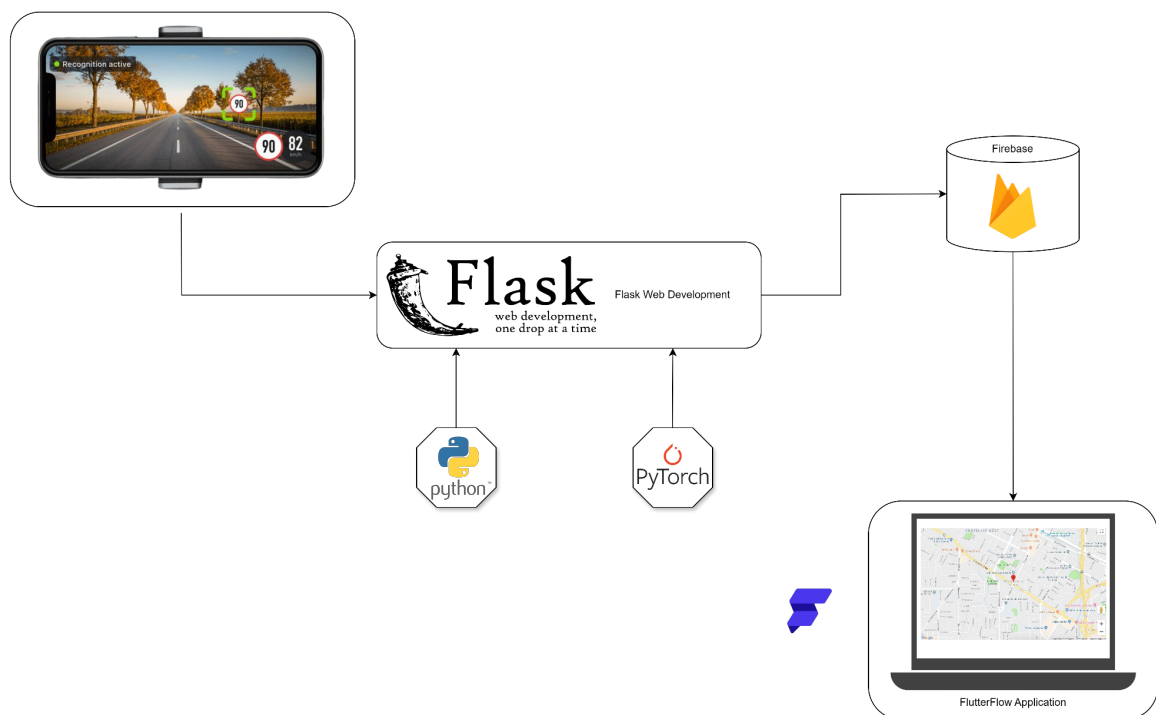


Figure 10. System Hardware Resources

The objective is to train a fast-inferencing and accurate neural network and deploy it for a backend flask server successfully. To do this, as we can see in Figure 10, will be taking advantage of the powerful advancements of **Python** in the Deep learning, and Computer Vision field and utilize it as the main programming language for our project. Alongside, **PyTorch** would serve as the primary framework to assist us in building these models. The **YOLOv3** deep learning architecture is the primary architecture we aim to train our model on

as it tends to give the greatest accuracy with limited or restricted resources. However, to train the model effectively, a considerable amount of data is essential. The dataset of the **US Traffic Roadway images** provided is quite niche and limited. To increase the number of data in our dataset, we performed data augmentation techniques using Imgaug. The primary object of our project is to be able to detect and localize Traffic Signs while ensuring that unique annotations persist. The YOLOv3 model we aim to develop would be optimal as it is both fast and accurate for object detection. Next, to capture images of the Traffic signs, we would require a smartphone camera that sends the images to the Traffic Sign processing pipeline to continue the main process. We decided to host the deep learning model on the **Flask web framework** as it provides a backend for the communication to the **FlutterFlow application**. The backend would deduplicate images in a sequence and based on the distance of the detected Traffic Sign obtained from the deep learning model, the geolocation of the traffic sign can then be calculated. Finally, the traffic signs obtained and their respective geolocation will be transmitted to our cloud service provider, **Firebase cloud service**. Firebase functions as the hosting platform for accessible data through user interfaces. Given its identity as a cloud server, Firebase empowers users to remotely access, modify, and append data to the system.

5.3 Alternative Designs

Deploying the model completely on the cloud would incur costs associated with server rentals and ensuring sufficient bandwidth. Nevertheless, leveraging cloud services presents the advantage of extending the solution's accessibility to multiple locations and entrusting system maintenance to a specialized third party.

Since we are dealing with the localization of the detected traffic signs, we need to employ a suitable database to store the geolocation in an appropriate manner. Since we will mostly be reading to the database, the type of database chosen must allow fast and efficient reads from the database. In addition to these, the writing of the data concerning predicted traffic signs and their respective geolocation when a user uploads images must be fast and should provide a gateway to efficiently be mapped to the Google maps API in the user interface where the users can view the pinned locations of the detected traffic signs. Table 2 shows the comparison of different types of databases.

Approach	SQL	No-SQL	GIS
Nature	Relational databases	Non-relational, diverse data models	Geographic Information System databases specifically designed for spatial data
Structure	Tables with predefined schema	Schema-less	Stores and organizes spatial data
Queries	SQL for well-structured, normalized queries	Uses various query languages, flexible	Spatial queries for geographic analysis
Scalability	Vertical scaling	Horizontal scaling	Depends on GIS software and tech.
Flexibility	Structured and rigid data model	Dynamic schema, suitable for evolving data	Handles spatial data types and relationships.
Services	ACID transactions	Eventual consistency	Specialized spatial indexing, geoprocessing
Examples	MySQL, PostgreSQL	MongoDB, Cassandra	PostGIS (extension of PostgreSQL), GeoServer

Table 6: Comparison of Types of Databases

Based on the comparisons shown in Table 6, we would prefer to use a mix of NoSQL and GIS type databases since we require the spatial data to be stored and retrieved effectively. However, since we only plan to store the geolocation and not intend to perform advanced GIS functionalities like spatial analysis, we would use NoSQL for our database. The reason being, it is an extensive database that is cloud hosted, enabling housing of traffic sign data.

Next, we would have to employ this database in a cloud server that houses the traffic sign data. Table 7 shows the comparisons of the different cloud database servers and its compatibility with PostGIS.

Approach	Microsoft Azure Database	Google cloud SQL	Amazon RDS	Firebase
Compatibility	Compatible with PostGIS	Compatible with PostGIS	Supports PostGIS	Supports PostGis
Performance	Provides different performance tiers	Depends on configuration	Depends on chosen configuration	Low latency
Scalability	Based on workload requirements	Provides options for vertical caling and offers high availability configurations	RDS provides options for scaling both vertically (instance size) and horizontally (Read replicas)	Scales seamlessly
Pricing	Basic, General purpose, Memory optimized	Instance pricing CPU and memory pricing Storage pricing and Memory pricing	Pay for what you use	Free to use within a limit

Table 7: Comparison of Types of Cloud Database Services

From Table 7, it is not very simple to determine the best cloud database service. Each of the top service providers boast competitive qualities. However, based on user reviews and general feedback, we determined that Firebase was the best choice. Furthermore, it also provides an effective gateway to the Google Maps API, being part of the Google Suite of products. We also chose to use Firebase as it offered the best pricing for performance and storage.

Finally, to develop a user interface system, we have to choose a framework that we would use to develop a user friendly as well as an extensible User Interface. Table 8 offers a comparative analysis of four prominent frameworks with libraries supporting integration with the cloud database service provider. While all four frameworks are valid options, our inclination is towards the adoption of Flask and, potentially FlutterFlow for application development purposes. This choice is influenced by several factors, including the robust documentation associated with FlutterFlow, its alignment with Firebase’s ecosystem, and the seamless integration provided by Flask.

FlutterFlow, known for its beginner friendly and no-code development platform, facilitates easy application development. Its seamless integration with the Firebase ensures a cohesive and well-supported database schema. Integrating Flask into the backend development stack further strengthens our choice since Flask’s. Flask's lightweight and flexible nature make it an ideal choice for building scalable and customizable backend solutions. With its simplicity and

versatility, Flask empowers developers to design RESTful APIs, handle complex business logic, and manage database interactions with ease.

Feature	JavaScript	Node.js	Vue	FlutterFlow
Library/Framework	JavaScript Library	JavaScript Runtime	JavaScript Framework	UI Builder/No-code Platform
Type	JavaScript, TypeScript	JavaScript	JavaScript, TypeScript	Dart
UI Paradigm	Component-Based	Event-Driven	Component-Based	Component-Based
Performance	Fast	Fast (Non-blocking I/O)	Fast	Fast
Native Mobile Support	Requires React Native	No native support	Vue Native	Yes
State Management	Redux, Context API	Express and many more	Vuex	Flutter State Management
Learning Curve	Steeper for beginners	Moderate	Gentle for beginners	Easy for beginners

Table 8: Comparison of Frameworks

Opting for a combination of FlutterFlow and Flask for application development offers a cohesive full-stack environment.

6. Project Management Plan

August 2023 - November 2023:

1 Project research and literature review						
1.1	Ideation - Research on potential topics	4-8-23	27-8-23	23	Aadhith, Koushal, Raaed, Vibha	100%
1.2	Finalize Project Topic with Advisor	1-9-23	2-9-23	1	Koushal, Aadhith	100%
1.3	Identify Project Stakeholder (iEngineering)	1-9-23	2-9-23	1		100%
1.4	Research and Summarize Previous Work Done	1-9-23	22-9-23	21	Vibha, Raaed	100%
2 Project Planning						
2.1	Problem Statement and Design Objectives	22-9-23	27-9-23	5	Raaed, Aadhith	100%
2.2	Project Management Plan Draft	28-9-23	5-10-23	7	Vibha	100%
2.3	Draft of Literature Review	5-10-23	8-10-23	3	Aadhith, Koushal, Raaed, Vibha	100%
2.4	Draft of URG Proposal and Submission	5-10-23	10-10-23	5	Aadhith, Vibha, Koushal	100%
2.5	Research of the Required Hardware and Tools	9-10-23	12-10-23	3	Raaed	100%
2.6	Submission of Report I: Introduction, Problem Statement and Design Objectives, Literature Review	10-10-23	12-10-23	2	Aadhith, Koushal, Raaed, Vibha	100%
2.7	Submit hardware purchasing request to Prof.Wissam	10-10-23	10-10-23	1	Koushal	100%
3 System Design and Functional Requirements?						
3.1	Meeting with iEngineering	12-10-23	12-10-23	1	Aadhith, Koushal, Raaed, Vibha	100%
3.1.1	Understand the Functional Requirements from the Stakeholder	12-10-23	14-10-23	2	Vibha, Aadhith	100%
3.2	Design the System Architecture	12-10-23	19-10-23	7	Koushal, Raaed	100%
3.3	Submission of Report II: Requirements, Cost Estimate, Updated Project Management Plan, Social Impact	15-10-23	19-10-23	4	Aadhith, Koushal, Raaed, Vibha	100%
4 Project Modeling						
4.1	Define a Testing and Validation Plan	20-10-23	28-10-23	8	Raaed, Vibha	100%
4.2	Tabulate and Analyze the Results from Literature review	20-10-23	28-10-23	8	Raaed	100%
4.3	Submission of Report III : System (SW/HW) Architecture and Detailed Design	2-11-23	2-11-23	1	Aadhith, Koushal, Raaed, Vibha	100%
4.4	Work on the Result Analysis with Standards	4-11-23	9-11-23	5	Raaed, Vibha	100%
4.5	Submission of Report IV: Verification, Validation, and Testing Plan and Result Analysis along with Standards	9-11-23	9-11-23	1	Aadhith, Koushal, Raaed, Vibha	100%
4.6	Draft of Final Project Report Submission to the Advisor	13-11-23	13-11-23	1	Aadhith, Koushal, Raaed, Vibha	100%
4.7	Final Project Report Discussion and Submission	13-11-23	16-11-23	3	Aadhith, Koushal, Raaed, Vibha	100%
4.8	Final Presentation for Senior Design I	20-11-23	22-11-23	2	Aadhith, Koushal, Raaed, Vibha	100%

Figure 11. List of Tasks, August to November

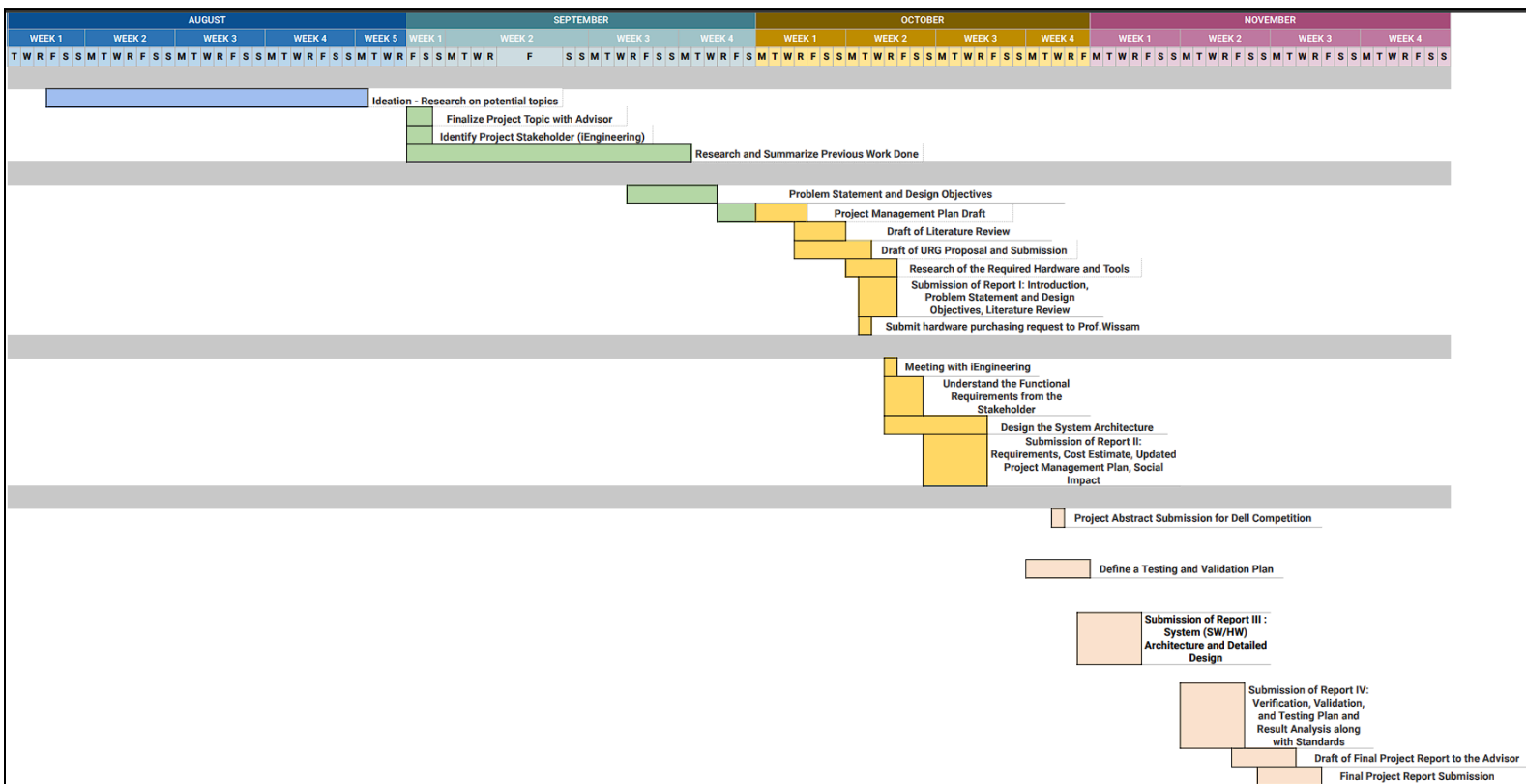


Figure 12. Gantt Chart

January 2024 - May 2024:

5 Implementation						
5.1	Perform data cleaning and analysis	23-1-24	2-2-24	10	Aadhith, Koushal, Raaed, Vibha	100%
5.2	Train the YOLOv5 and YOLOv3 on the dataset	23-1-24	27-2-24	35	Aadhith, Koushal, Raaed	100%
5.3.1	Develop a method to remove duplicate traffic sign instances	8-2-24	27-2-24	19	Aadhith, Koushal	100%
5.3.2	Develop a method to calculate the geolocation of the traffic signs	8-2-24	27-2-24	19	Raaed, Vibha	100%
5.3.3	Implement a technique that groups sequences of images	15-2-24	27-2-24	12	Aadhith, Koushal, Raaed	100%
5.4	Analyse the training results	22-2-24	27-2-24	5	Aadhith, Koushal, Raaed	100%
5.5	Project Management Plan and Standards	22-2-24	23-2-24	2	Aadhith, Koushal, Raaed, Vibha	100%
5.6	Address class imbalance in the dataset and perform any additional feature engineering	27-2-24	7-3-24	9	Aadhith, Koushal, Raaed, Vibha	100%
5.7	System (SW/HW) Architecture and Detailed Design	1-3-24	9-3-24	8	Aadhith, Koushal, Raaed, Vibha	100%
5.8	Implement the de-duplication technique and optimize the training results	9-3-24	27-3-24	18	Aadhith, Koushal, Raaed, Vibha	100%
6 Testing						100%
6.1	Acquire UAE traffic signs dataset	1-4-24	15-4-24	14	Aadhith, Koushal, Raaed, Vibha	100%
6.2	Assemble the hardware acquired	7-4-24	13-4-24	6	Aadhith, Koushal, Raaed, Vibha	100%
6.3	Test the model on the UAE traffic signs	13-4-24	18-4-24	5	Aadhith, Koushal, Raaed, Vibha	100%
6.4	Verification, Validation, and Testing Plan and Result Analysis	15-3-24	20-3-24	5	Aadhith, Koushal, Raaed, Vibha	100%
6.5	Analyse and document the testing results	22-4-24	25-4-24	3	Aadhith, Koushal, Raaed, Vibha	100%
7 Presentation						100%
7.1	Draft Final Report submission to Advisors	25-4-24	28-4-24	3	Aadhith, Koushal, Raaed, Vibha	100%
7.2	Final Report Submission	30-4-24	2-5-24	3	Aadhith, Koushal, Raaed, Vibha	100%
7.3	Final Presentations and Demos	5-5-24	7-5-24	2	Aadhith, Koushal, Raaed, Vibha	0%

Figure 13. List of Tasks, January to May

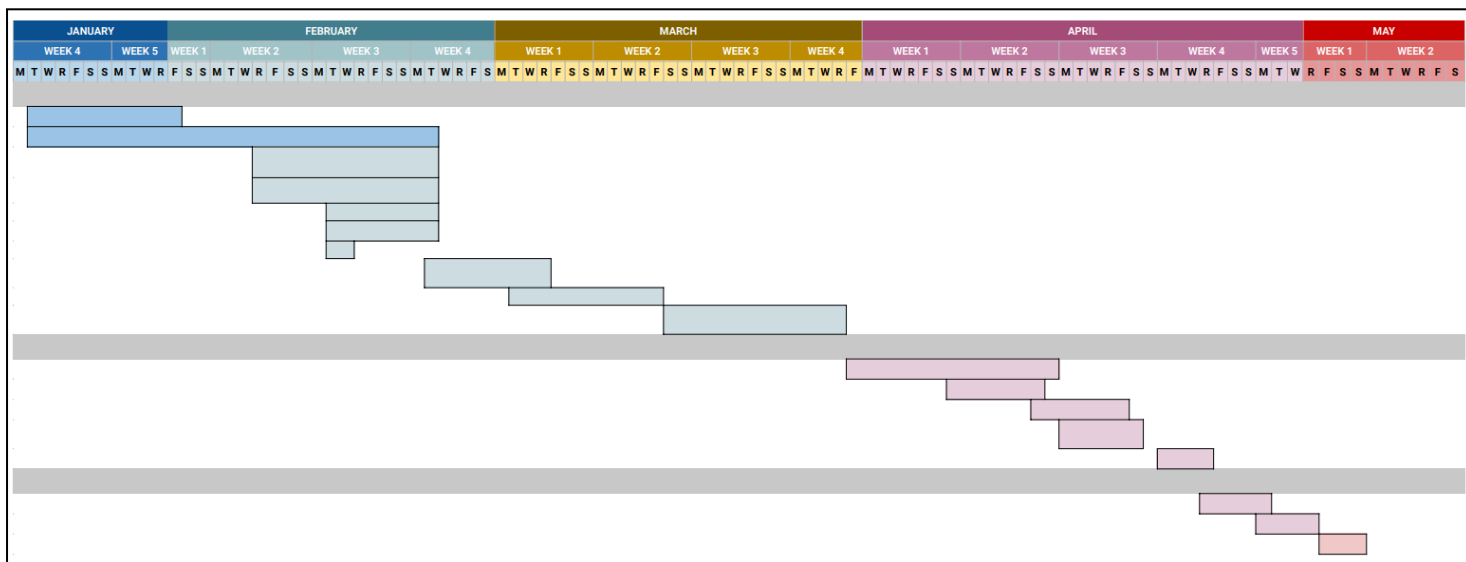


Figure 14. Gantt Chart

7. Implementation

7.1 Dataset Analysis

7.1.1 Key findings from the dataset

The dataset was procured from our stakeholder iEngineering PVT LTD. It contains:

1. Images of roadways in the city of Bowie, Maryland, U.S.A (67487 images)
2. Cropped traffic signs in the dataset distributed among 320 folders each representing a traffic sign class (320 classes)
3. Bounding box coordinates of detected traffic signs detailed in a file titled 'City of Bowie - TS - R1_2.csv'
4. Metadata of the detected traffic signs in a file titled 'City of Bowie - Traffic Signs - R2_2.csv'
5. XML files of the bounding box coordinates of all the detected traffic signs

The initial steps included manually inspecting the dataset of the city of Bowie, which were frames extracted from videos of the roadways. The frames were taken from 9 videos and hence the dataset was split into 9 folders containing frames from the corresponding videos.

This was necessary for the development of the optical flow algorithm. Upon further investigation, we recognized that out of the complete dataset, we had labels for a small sample. This also resulted in an imbalance in the dataset, leading to unequal representation of the 320 classes for the model training ranging from 2710 images to just 1 image representing a class. This would be detrimental to the performance of the model as it would not be trained enough on the underrepresented and perform poorly when testing on those classes (lower accuracy, F1 score, etc.).

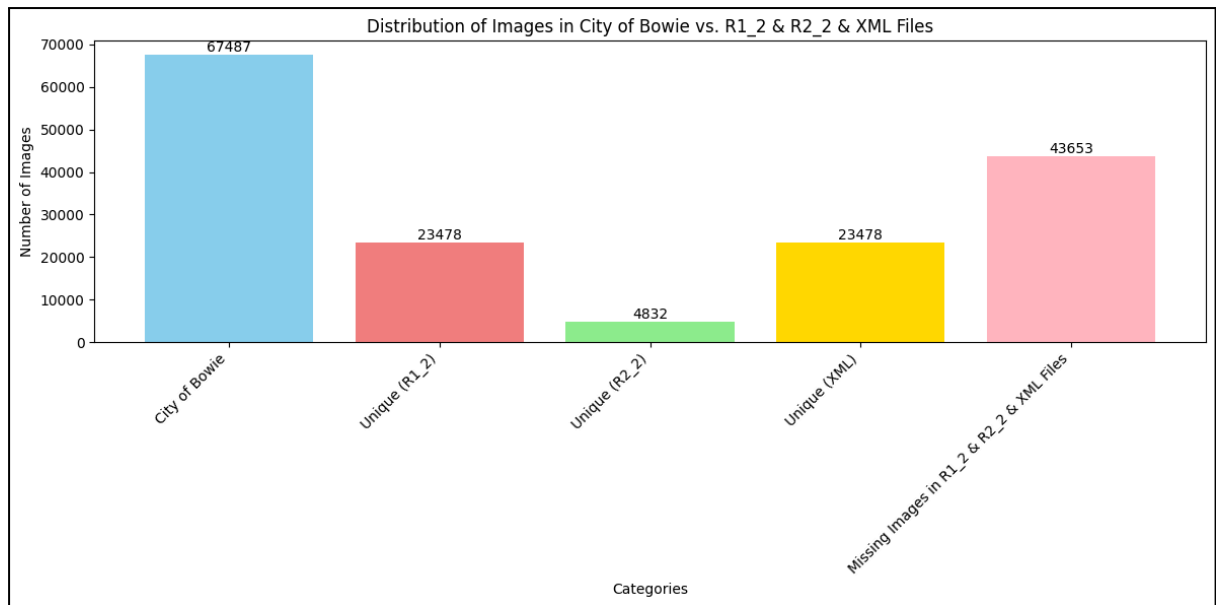


Figure 15: Distribution of unique images in the dataset

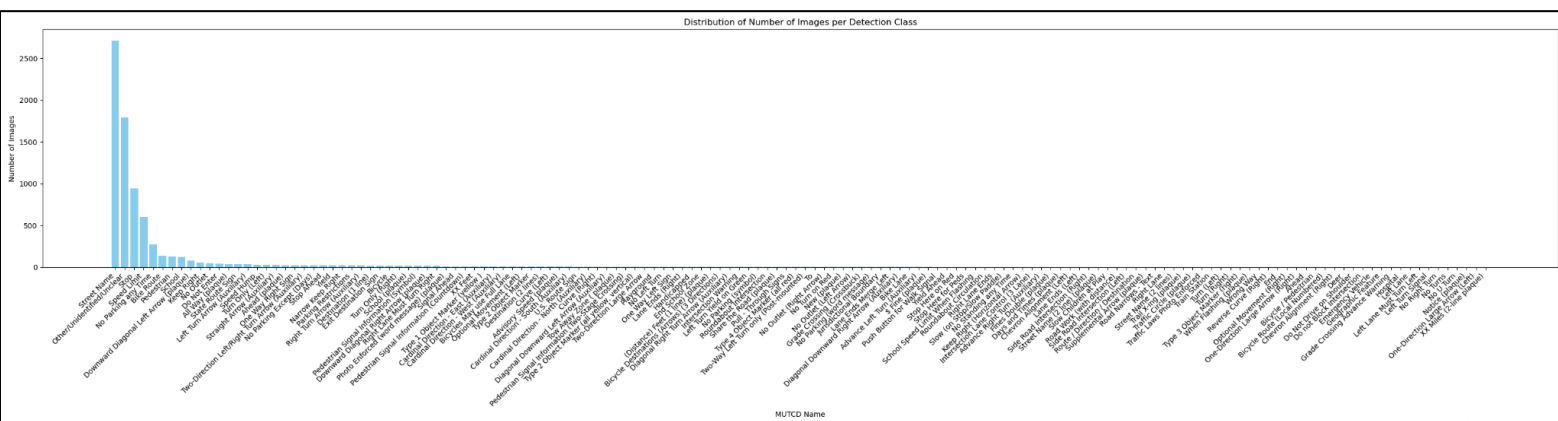


Figure 16: Class distribution Histogram

The labels in the City of Bowie - Traffic Signs - R2_2.csv file also contained classes apart from the 320 classes. It was compiled into a single class called “other” and was excluded since they were not compliant with MUTCD traffic signage. Not all 320 classes had representative images in the labeled dataset and therefore, the actual number of classes that were available for training were fewer.

	Classes	Images	Missing Classes
Traffic Sign Labeled Crops	320	48747	16
R2_2	148*	4476	188

*Note: Excluded some non-MUTCD classes that started with 'other', or was 'unidentified', 'unclear'

Table 9: Class Reduction

To summarize, the deep learning model had 148 classes to train on and 4476 images that represented those classes.

7.2 Software

7.2.1 The Deep Learning Model

Upon noting the requirements from our client, iEngineering Private Limited, for the tools and models to be used for our project, we have decided to adopt the following:

- **Python** as the main programming language for our project brings forth numerous advantages in the realms of Deep Learning and Computer Vision. Python's extensive libraries, such as TensorFlow and PyTorch, provide a robust foundation for implementing sophisticated deep learning models. Moreover, Python's versatility facilitates seamless integration with computer vision libraries like OpenCV. Leveraging these powerful advancements in the Python ecosystem empowers our project with state-of-the-art capabilities in deep learning and computer vision.
- **PyTorch** as our primary framework streamlines deep learning model developments. The framework is built to speed up the process between research prototyping and deployment. Its widespread adoption and community support ensure access to cutting-edge advancements, empowering our project with efficient model construction, training, and deployment. It is also supported by major cloud platforms
- **YOLOv3** would be the primary deep learning model that we plan to implement and test. This is based on the results obtained from the literature review. Furthermore, our client iEngineering preferred the development of a YOLO deep learning model that would be easy to integrate into their existing transportation management system.

7.2.2 Optical Flow Algorithm for Deduplication of Traffic Signs

Optical flow, a method for estimating motion between consecutive video frames, aids in distinguishing the apparent velocity of traffic signs. By utilizing the optical flow constraint equation, one can compute the image velocity components (u, v) aligned with the image intensity gradient. This process identifies key features such as edges and corners of the traffic sign, which can then be tracked across frames using techniques like the Lucas-Kanade method. We use this method as it selects a sparse set of feature points and tracks their velocity vectors, enabling the prediction of the Region of Interest (ROI) where the traffic sign is likely to appear in subsequent frames. Leveraging this dense optical flow information, precise sequences of the traffic sign's movement can be generated, serving as valuable data for evaluating traffic sign recognition models. This is an essential process because images can contain multiple traffic sign classes in the same image, and sometimes, multiple signs of the same class in the same image. This algorithm can differentiate the traffic signs in the same image and create sequences of each traffic sign class and deduplicate the predictions such that only one reference image and prediction exist. This would also further optimize the geolocation calculation algorithm as it would calculate the geolocation of each sign class only once, thereby reducing overall inference time. Hence, we ensure the uniqueness of annotations that persist in the database.

7.2.3 Localization of the Detected Traffic Sign

The second main objective of this project is to obtain a precise estimate of the signs' geolocation after the model detects the various traffic signs. The algorithm for calculating the geolocation consists of two steps, where the first step is to calculate the distance between the camera and the detected sign and the second step is utilizing the haversine formula to find the latitude and longitude of the detected sign.

Firstly, the calculation of the distance between the camera and the traffic sign is dependent on the real height of the object, the focal length and image height in terms of pixels based on the bounding box formed upon detection of the traffic sign. Additionally, the sensor height of the camera used is required to calculate the distance to the object. [41]

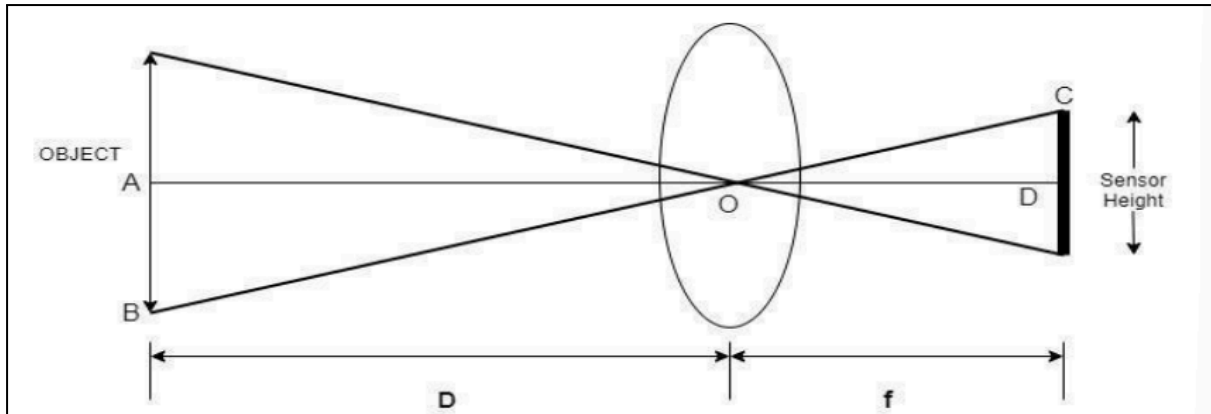


Figure 17: Distance from Camera to Object Detected [41]

$$Distance\ to\ Object = \frac{Real\ height\ of\ Object\ (mm) * Camera\ Frame\ Height\ (pixels) * f\ (mm)}{Image\ Height\ (pixels) * Sensor\ Height\ (mm)} \quad (1)$$

This formula is based on the lens principle, where the distance (D) is calculated using the focal length (f) as well as the height of the object, sensor and image. Once this distance has been calculated, the formula to determine the latitude and longitude coordinates of the sign is based on the haversine formula in Figure 18, where δ denotes the relative distance, ψ is latitude, λ is longitude, and R is the mean of earth's radius: 6,371 km.

$$a = \sin^2\left(\frac{\Delta\psi}{2}\right) + \cos\psi_1 \cdot \cos\psi_2 \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right),$$

$$\delta = 2R \cdot \text{atan2}\left(\sqrt{a}, \sqrt{1-a}\right).$$

Figure 18: Haversine Formula [42]

$$latFactor = \frac{distance}{111000} \quad lonFactor = \frac{distance}{111000 * \cos(latitudeRadians)}$$

The latitudeRadians is latitude in radians and the distance is in meters. 111000 denotes the approximate distance in meters of one degree of latitude.

Hence, the new latitude and longitude are calculated are:

$$newLatitude = oldLatitude + latFactor \quad (2)$$

$$newLongitude = oldLongitude + lonFactor \quad (3)$$

Therefore, by deriving the formula for the new latitude and longitude from the haversine formula, it can be calculated with the use of old latitude and longitude, and the distance to the object from the camera.

7.2.4 Front-end - FlutterFlow Application for the User Interface

The front-end of the user interface website must be dynamic and easily accessible on a desktop as well as mobile devices such as smartphones and tablets. For this reason, the front end will be developed using FlutterFlow and involves utilizing the power of Dart for app development.

The objective of the application is to render a user interface that displays the information of the traffic signs detected. This is done by retrieving the data of traffic signs and their respective geolocations and efficiently mapping them into the Google Maps API. The traffic signs would be pinned to the maps and upon clicking the pin, the user gains access to the specific details about the sign such as the type and the geolocation.

7.2.5 Back-end

The backend for the user interface of the FlutterFlow application will be developed using Flask, a Python web framework. Flask provides a robust and flexible environment for building FlutterFlow applications, offering a wide range of extensions and libraries that streamline development. It excels in creating RESTful APIs, handling database operations, and integrating with various services like Firebase cloud services for authentication and data storage. Flask also provides a pipeline to remotely deploy our YOLOv3 model, Optical Flow algorithm, and the Geolocation calculation algorithm, while also handling input and output of data from the FlutterFlow application and to Firebase respectively.

Firebase is the cloud service chosen as the backend for storing and retrieving data. Firebase contains 3 main components utilized in our system architecture- User Authentication, Firebase Storage, Firestore Database. User Authentication is a feature utilized by our front-end to authenticate the credentials of the users logging into the application. Firebase Storage is used to store the images of the predicted traffic signs. Firestore database stores two databases: the user database consisting of the user login details, and more importantly, the image_details database that comprises the data concerning the predicted traffic signs and a URL to the respective image in the storage.

Flask, thus, leverages Firebase's authentication services, real-time database capabilities, and cloud functions to enhance the functionality and user experience of the FlutterFlow application. Together, Firebase and FlutterFlow provide a comprehensive solution for building FlutterFlow applications with robust backend functionality and secure data management.

7.3 Integration

The overall solution of the project integrates all the above mentioned software components - Flask, FlutterFlow, and Firebase. The Flask backend server utilizes the RESTful API to serve as a bridge between Firebase and FlutterFlow. Together, these 3 main software components work to detect traffic signs in roadways, obtain their geolocations, store these predicted images, and localize them on a map. From a smartphone or any other device, users can upload the images of roadways collected from a smartphone through the FlutterFlow application.

Capturing images from a smartphone is a prerequisite since the images are tagged with their respective geolocation of when the image was captured as well, which is critical to locate the traffic signs in our application. These images are sent to the backend Flask server through a POST API call ('/sendImage'). The Flask server hosts the critical components of our system: trained YOLOv3 object detection model, Optical Flow algorithm, and Traffic Sign Geolocation calculation algorithm. In the Flask server, the EXIF details of the received images are extracted and the images are then pre-processed to be compatible with the YOLOv3 model and the model detects traffic signs present in the images (if any). The predictions from the YOLOv3 model undergoes Optical Flow to deduplicate traffic signs detected multiple times at the same location. Based on the captured predicted bounding box of the class of traffic sign, the EXIF details of the image, focal length of the camera, real height of that class of traffic sign, and height and width of the image, the geolocation of the predicted traffic signs are then calculated. Finally, the data concerning the deduplicated predicted traffic signs and their respective geolocations are grouped together and sent to the Firebase database.

The connection between Firebase and FlutterFlow is facilitated through the Firebase SDK. FlutterFlow utilizes the Firebase SDK to access the Firestore database in Firebase. The data concerning the detected traffic signs and their precise geolocations is retrieved by the

FlutterFlow app from the Firestore database, enabling the app to obtain and display this information. The data is sent utilizing the Firebase admin SDK through the HTTP protocol. On FlutterFlow, users can view pins on a map depicting the location of the detected traffic signs and upon clicking, the users can view details regarding the detected class of the traffic sign and the precise geolocation of that traffic sign as well.

8. Verification, Validation, and Testing Plan and Result Analysis

8.1 Metrics

Determining the metrics used to analyze the performance of the deep learning model used in this project is essential. By establishing a clear outline for the evaluation, it would gain insights into the model's strength and compare it to the existing models working on a similar problem. The validation and verification can be done on the traffic sign detection model through the use of the following metrics:

1. Intersection over Union (IoU)

IoU is a metric used to evaluate object detection models. This metric quantifies the degree of overlap between two regions/bounding boxes, and it rewards the model for predicting bounding boxes that are heavily overlapping with the ground truth bounding box (the region where the object actually is).

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of union}}$$

Dividing the area of overlap between the predicted bounding box and the ground truth box by the union of both boxes results in the IoU score for that prediction. The IoU score ranges from 0 to 1, with 1 being a perfect prediction.

2. Confusion Matrix

The confusion matrix is a very common performance measurement used for classification problems in machine learning, and it is integral for calculating Precision and Recall. The confusion matrix is a table with 4 different combinations of predicted values (rows) and actual values (columns); the 4 combinations are True Positive (TP), False Positive (FP), False Negative (FN), and True Negative (TN).

Object detection is not a classification problem; however, by defining an IoU threshold that must be exceeded to consider a prediction correct, a confusion matrix becomes applicable to the model and the 4 combinations can be interpreted as follows:

- a. TP: The IoU threshold was exceeded, meaning an object was detected in that region which matches the ground truth.
- b. FP: The IoU threshold was not exceeded for a prediction.
- c. FN: Ground truth was not detected.

Note that TN is not applicable to object detection, as it is equivalent to not detecting anything.

3. Precision

It is the proportion of correct predictions (True Positives) to the sum of ground truth objects. Precision represents the percentage of results that are relevant (How good the model is at detecting objects).

$$P = \frac{TP}{(TP + FP)}$$

4. Recall

Recall is the proportion of the correct predictions (True Positives) to the sum of all predictions made. It represents the percentage of correct detection versus all detections (how often the model is correct).

$$R = \frac{TP}{(TP + FN)}$$

5. F1 Score

The F1 score is the harmonic mean of precision and recall, which helps us measure recall and precision at the same time. The use of recall and precision means we consider FP and FN making it a more relevant metric than simple accuracy. The harmonic mean is used instead of an average as means of punishing extreme values.

$$F_1 = 2 * \frac{P * R}{(P + R)}$$

6. Average Precision (AP) and Mean Average Precision (mAP)

AP is the weighted mean of precisions at each IoU threshold, where the weight is the increase in recall from the previous threshold. Or simply, it is the area under a precision-recall curve. By calculating the mean of AP values over all IoU thresholds we result in the mAP score.

$$mAP = \frac{1}{N} \sum_{k=1}^{k=n} AP_K$$

This score is the current benchmark measurement used to evaluate computer vision models. It showcases the trade-off between precision and recall while maximizing the effect of each metric.

7. Average Inference Time

The average inference time of a model processing a set of images is a critical metric for evaluating its efficiency and performance. It signifies the average duration the model takes to make predictions on a given image set. This metric offers insights into the model's real-time applicability and responsiveness. Optimizing the model to reduce the average inference time can enhance decision-making speed, user experience, and overall system performance. Monitoring and understanding this metric are essential to ensure the model meets the necessary speed and responsiveness standards for its intended application.

8.2 Testing

8.2.1 White Box Testing

The user interface in this project is a FlutterFlow application, where the user can interact with the system and mainly view the detected signs and their geolocation. As this is a software application in this case, performing white box testing on this is crucial. This is done in order to evaluate the logic, code and the internal structure of the application. Some of the aspects of testing that was conducted was Load Testing. Below is the graph of total execution time versus load:

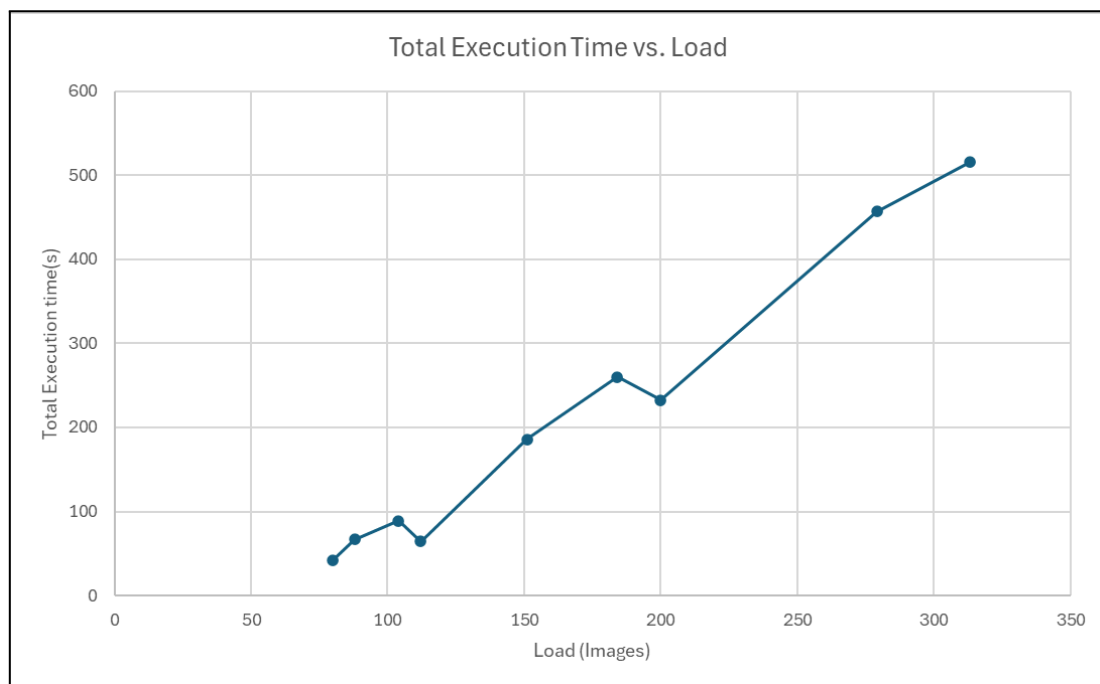


Figure 19: Total Execution Time versus Load

From Figure 19, we can see that as you increase the number of images (load) sent to the pipeline, the time it takes to process the images increases. The maximum load our pipeline could handle was around 400 images. We conducted a trial of 400 images loaded into the pipeline, and as a result, the pipeline crashed.

8.2.2 User Acceptance Testing (UAT)

This is a form of black-box testing where it validates the testing done after the final stage of development. This is mainly performed by the end users or stakeholders, where they interact with the system without any knowledge about the internal code or implementation details. This testing helps evaluate the users experience with the system and assesses if it is able to deliver the intended functionalities.

In this project, the User Acceptance Testing (UAT) was performed on a group of users who were surveyed upon their use of the system and attained their evaluation based upon their experience. To conduct the test, we sent out a Google Form to collect feedback from users based on their experience upon using the application. This approach allowed us to systematically gather and analyze user input on various aspects of the application. The Google Form included questions that covered different areas such as usability, functionality,

performance, and overall satisfaction. We received responses from nearly 100 people, which was effective in identifying important areas where the application works well and those that need to be improved. The input received via the Google Form yielded valuable feedback that will direct additional improvements to our application. Resolving the problems and adding user feedback will improve the application's overall quality and user satisfaction.

Question	Response
How comprehensive and well-organized did you find the information page on traffic signs and their MUTCD codes?	Overwhelmingly positive results
Did you find the application's interface responsive and fast in loading information and map data?	Positive responses
How would you rate the overall appearance of the application?	Overall positive results

Some of the recommendations received were to improve the application included enhancing the user interface by making improvements to the layout to enhance the usability and with the overall look of the application,

8.3 Results Analysis

The YOLOv3 and YOLOv8n models were trained and tested on the dataset to compare the results. YOLOv3 performed significantly better. Below is a table showing the results of the models

Models	Precision	Recall	F1-score	mAP_0.5	mAP_0.5:0.95
YOLOv3	96.60%	94.62%	95.60%	96.18%	81.31%
YOLOv8	91.20%	83.13%	86.98%	88.38%	71.92%

Table 10: Results of Model Testing

For the objective of ensuring uniqueness in the annotations, we developed a deduplication algorithm using optical flow. The algorithm requires 2 consecutive frames, using which we can estimate the apparent velocity of the region of interest (ROI) points, which are the x and y coordinates of the bounding box of the detected traffic sign acquired from the model

predictions, to extrapolate the predicted ROI points in the third and consecutive frames. In figure 20, we can see the visual representation of the apparent velocity or flow vectors of a traffic sign that appeared to move in consecutive frames. Hence, we used the predicted ROI points and the actual ROI points of the traffic sign and calculated the intersection over union (IOU) to determine whether a traffic sign detected in consecutive frames is the same sign in the sequence or not. We also verify whether the model predicted that the traffic sign in the subsequent frames is the same detection class to continue the sequence identification process. Once a sequence is identified and terminated when the traffic sign no longer appears in consecutive frames, we perform deduplication, where we keep one image with the highest confidence score for the respective traffic sign class to store the metadata in the Firestore database and the respective image in the Firebase storage.



Figure 20: Predictions of the ROI using Optical Flow

For the objective of precisely estimating the geolocation for the image, we attained geolocation of signs with a margin of error of ~ 6 meters compared to the ground truth consisting of lat-long coordinates. The distance between the detected sign and the smartphone's location was calculated using the haversine formula for the latitude and longitude using ground truth coordinates and estimated coordinates and was used to find the margin of error.

An example of this process for a single image having a **STOP** sign has the initial asset latitude and longitude as 39.0046994 and -76.77848053 respectively, and the object latitude and longitude from the ground truth is 39.0047 and -76.7783 respectively. The haversine

distance is then calculated to find the distance between latitudes and longitudes which was determined to be 15.59955005.

Using the formula mentioned above (1), the distance between the **STOP** sign and the smartphone's position is calculated using the height values of the real object, image, and sensor, which was estimated to be 10.17206929. This calculated distance was used to find the new latitude and longitude of the detected sign using formulas (2) and (3), which was 39.00479104 and -76.7783626 respectively. Finally, the new haversine distance is calculated using the latitude and longitude in the ground truth and the predicted ones, which was 14.41073596. The absolute difference between the two haversine distances calculated came to be 1.188814087. Hence, the margin of error in the calculated geolocation for this sign was around a meter. Similarly, after calculating the absolute error for all the images under the MUTCD Category 'Regulatory Signs', the average margin of error was found to be ~6 meters.

We have successfully deployed the model on Flask and integrated the three objectives to work sequentially. Moreover, we created an application where users can interact with location markers on a Google Maps API within a FlutterFlow environment. This allows users to view detected traffic signs in the city along with their geolocation. Additionally, we implemented functionality for users to upload images. Upon uploading, the application calculates the geolocation and places location markers for the detected traffic signs based on this information. Figure 21 represents the pinned locations of the detected traffic signs on the map. FlutterFlow obtains this data from the Firestore database in Firebase.



Figure 21: Pinned Location Markers displaying the detected signs on the Flutterflow app

Figure 22 depicts the data displayed upon clicking the location pin. The data regarding the detected traffic sign such as the Detection class, MUTCD name, Geolocation and the image containing the predicted traffic sign are displayed.

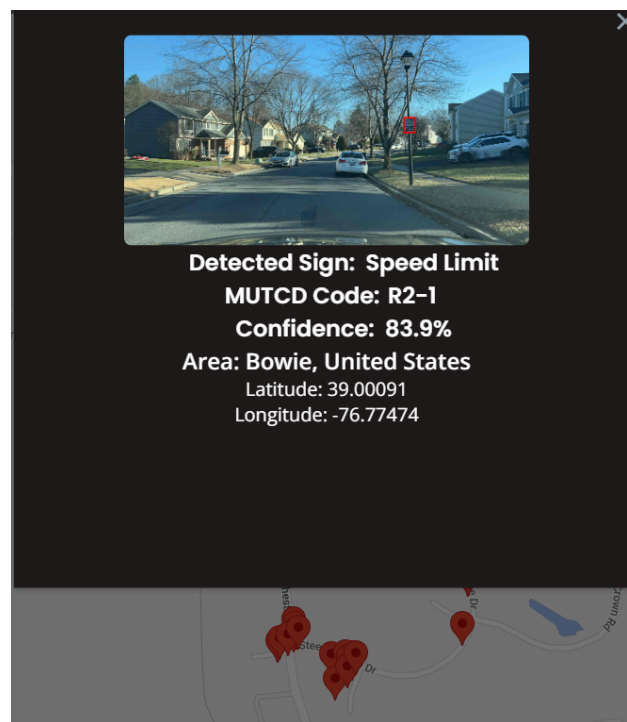


Figure 22: Traffic sign information displayed upon marker click

9. Project Global, Economic, Societal Impact

The proposed project holds a profound significance and advancement at the intersection of the fields of transportation and computer vision. We are focusing on traffic sign detection and localization. We believe this effort can produce valuable results that are essential in these areas, and it could also bring some fresh and clever ideas to both fields. The impacts of this project can be outlined as follows:

1. Transport Asset Management Service:

The essence of any transportation network is to ensure safety, efficiency, and reliability for all road users (including the pedestrians). The significance of this project lies in its dedication to optimizing the management of the assets constituting infrastructure and traffic signs and to road safety. The primary outcome of the project is to develop a product that will be integrated into iEngineering's existing Transportation Asset Management Service. Our project may also be integrated into the US Department of Transportation to serve as an improvement on existing technologies such as the sign detection and the integration of autopilot. By enhancing the capacity to extract roadway asset information from images and ensuring accuracy, real-time asset data can be instrumental in effective infrastructure management, safety, and decision-making in a SaaS format. It stands as a testament to creative problem-solving, pushing the boundaries of technology to meet the unique demands of the transportation sector.

2. Computer Vision in Transportation:

Developing an algorithm capable of traffic sign detection, recognition and localization is a unique challenge with broad applications in areas such as autonomous vehicles and smart transportation systems. Hence, this project's outcomes will extend the boundaries of the capability of computer vision in the field of transportation addressing some real-world challenges.

3. Practical Implementation and Industry Impact:

The integration of this system will revolutionize how transport assets are managed and maintained, allowing for optimal and efficient operations. The project's integration into an existing industry service line showcases the innovative approach to addressing an existing challenge in the field of Transportation, an approach to meet the current-industry demands.

Integrating the algorithm into iENGINEERING Private Limited's transportation asset management service demonstrates the project's immediate industry applicability.

In conclusion, the project will have a substantial impact at the intersection of computer vision and transportation asset management service. Our project offers improved asset inventory capabilities, innovative approaches to practical issues, and the potential to revolutionize the transportation sector by enhancing safety and upkeep of infrastructure. Through thorough research and development of our project, the work aims to bridge the gap between theory and practice, making substantial contributions in the field of Computer Vision and Transportation.

10. Standards

When it comes to the field of software development, standards are a set of established specifications and guidelines that ensure consistency and enforce the development of quality software. These rules use consistent metrics for evaluation of projects across the software industry, and improve their credibility and reliability.

Since the proposed project uses a dataset from iEngineering, which is a US based company, it is essential that our system follows standards made for the US. Additionally, as the proposed system will be tested in the UAE, it is important to comply with the UAE standards as well. The standards to be followed in the context of this project are related to the traffic regulations in both of the countries.

The standards for traffic signs in the US as well as other traffic control devices are written in the *Manual on Uniform Traffic Control Devices* (MUTCD). This manual also consists of guidelines for the lettering used for such signs on the highway and the specifications for the color used for these as well [36]. The manual highlights the importance of the color used as an indicator of the type of information it contains. For instance, the color red is used to indicate prohibitory signs and the color orange indicates warning or roadway work zone markings. Hence, it is followed to ensure that the roadway users are alerted in a uniform manner, which can be comprehended by the general public.

The ISO standards are a set of internationally recognized standards which is also created to maintain consistency in the projects worked on in the industry. The standards required in this project are:

1. **ISO 14819-1:2003 - Intelligent transport systems - Traffic and travel information messages via traffic message coding [37]**

This standard is applicable to transport systems where there is geographic data, and road signs and symbols are involved.

2. **ISO 21448:2019 - Road vehicles - Safety of the intended functionality [38]**

This standard deals with the safety of intended functionality in road vehicles. In this project, it is relevant due to the traffic sign detection system being integrated into vehicles.

3. ISO 28564-3:2019 - Public information guidance systems - Part 3: Guidelines for the design and use of information index signs [39]

This standard specifies all the guidelines for design, inspection and updating the information index signs used in public places.

Furthermore, the standards in the UAE differ from that of the US. Here, there are different regulations for each emirate in the UAE. The Quality and Conformity Council (QCC) in Abu Dhabi is a government entity that is established to raise the quality of Abu Dhabi's products [40]. They have developed certain standards and specifications in many aspects of the emirate, one of them being the standard which addresses the road network and signage. The *Abu Dhabi Emirate Standard for Signage* is a set of standards that provide guidelines to establish a homogeneous addressing for signage throughout Abu Dhabi.

Additionally, the road symbols and signage in Dubai is managed by the Roads and Transport Authority (RTA). This authority is responsible for the maintenance of these road signs, which depict the necessary traffic regulations. Hence, this project aims to comply with all the aforementioned standards.

11. Conclusion

11.1 Summary

Traffic signs are an integral part of implementing safety rules for drivers on roadways. These signs provide drivers with crucial information that helps maintain a decree and decrease possible cases of accidents. Hence, it is of utmost importance that these signs are ensured to be accurate and intact. As manual inspection of these signs would be time consuming and labor intensive, it is also prone to errors. It is crucial that the damaged signs are detected at the earliest as it would lead to potential hazards which poses a significant safety issue. This is where the need for traffic sign detection systems would come in place, where the safety of drivers would be ensured by an automated system. Hence, by implementing a system with a deep learning model, the detection and localization of traffic signs would be performed on an autonomous system, which will be tested using data from the client, iEngineering. The proposed system would capture the images of the roadways and retrieve the geolocation, which is then fed into the Flask server. This is where the deep learning model is implemented to detect and localize the traffic signs, which are then stored on firebase. This data can then be displayed for end users on a user interface such as a FlutterFlow application. The benefits of such a system include the determination of geolocation of the detected signs, which can be used by traffic management service providers who aim to strengthen the road safety conditions. Without manual inspection, an automated system which performs the above would aid in easing the process while also avoiding any potential hazards on the road in a prompt manner.

11.2 Future Work

Following the progress made till now, the next stage of our project could involve testing on edge devices and developing the traffic sign recognition system to perform in real time. Rigorous testing and optimization of the developed traffic sign recognition system is necessary for deployment on edge devices. This phase will involve extensive experimentation to ensure the robustness and efficiency of the system in real-world scenarios.

1. Testing on Edge Devices

The transition to edge devices is a pivotal step in our project's evolution, as it challenges the adaptation to the computational constraints typical of such hardware. Our team will conduct thorough testing on selective edge devices, including but not limited to the NVIDIA Jetson Orin Nano, to assess the system's performance under different conditions. This testing phase will focus on evaluating key metrics such as inference speed, resource utilization, and accuracy to ascertain the system's suitability for real-time deployment.

2. Real-Time Inference

Achieving real-time inferencing could further evolve our traffic sign recognition system, especially considering its potential applications in autonomous vehicles and transportation management. As such, the upcoming phase will prioritize optimizations aimed at minimizing inference latency and maximizing throughput without compromising accuracy. Algorithmic optimizations, and hardware acceleration will be explored to streamline the system's operation and enhance its responsiveness in real-time scenarios.

3. Training on more robust datasets

Currently, the system can achieve the objectives on controlled environmental conditions. However, such systems should be versatile in use and adapt to any external environment. Versatility is key for ensuring the effectiveness of our system across a wide range of real-world scenarios, where environmental factors such as lighting conditions, weather phenomena, and occlusions may significantly impact its performance.

4. Diverse model selection and benchmarking:

To ensure a comprehensive evaluation, we will test multiple models, considering a diverse range of architectures. This may include exploring other YOLO family models such as the YOLOv5 and CNN models like ResNet, MobileNet, MaskRCNN, FastRCNN and so on. By evaluating models with varying complexities, parameter sizes, and computational requirements, we aim to identify the most suitable candidate for our specific task and deployment environment.

References

- [1] Dewi, C., Chen, R.C., Jiang, X. et al. Deep convolutional neural network for enhancing traffic sign recognition developed on Yolo V4. *Multimed Tools Appl* 81, 37821–37845 (2022). <https://doi.org/10.1007/s11042-022-12962-5>
- [2] Dewi, C., Chen, R.-C., Liu, Y.-T., Jiang, X., & Hartomo, K. D. (2021). Yolo v4 for advanced traffic sign recognition with synthetic training data generated by various gan. *IEEE Access*, 9, 97228–97242. <https://doi.org/10.1109/ACCESS.2021.3094201>
- [3] Tang, Y., Yu, B., Wang, A., & Liu, F. (2023). A traffic sign recognition method based on yolov5 deep learning algorithm. 2023 The 7th International Conference on Machine Learning and Soft Computing (ICMLSC), 150–156. <https://doi.org/10.1145/3583788.3583810>
- [4] X. Liu, X. Jiang, H. Hu, R. Ding, H. Li and C. Da, "Traffic Sign Recognition Algorithm Based on Improved YOLOv5s," *2021 International Conference on Control, Automation and Information Sciences (ICCAIS)*, Xi'an, China, 2021, pp. 980-985, doi: 10.1109/ICCAIS52680.2021.9624657.
- [5] J. Hu, Z. Wang, M. Chang, L. Xie, W. Xu, and N. Chen, "PSG-Yolov5: A Paradigm for Traffic Sign Detection and Recognition Algorithm Based on Deep Learning," *Symmetry*, vol. 14, no. 11, pp. 2262–2262, Oct. 2022, doi: <https://doi.org/10.3390/sym14112262>.
- [6] C. Li and C.-Y. Su, "Traffic Signs Detection Based on Enhanced YOLOv5 Network Model," *2022 IEEE International Conference on Consumer Electronics - Taiwan*, Jul. 2022, doi: <https://doi.org/10.1109/icce-taiwan55306.2022.9868992>.
- [7] H. Zhang et al., "Real-Time Detection Method for Small Traffic Signs Based on Yolov3," *IEEE Access*, vol. 8, pp. 64145–64156, Jan. 2020, doi: <https://doi.org/10.1109/access.2020.2984554>.
- [8] L. Jiang, H. Liu, H. Zhu, and G. Zhang, "Improved YOLO v5 with balanced feature pyramid and attention module for traffic sign detection," *MATEC web of conferences*, vol. 355, pp. 03023–03023, Jan. 2022, doi: <https://doi.org/10.1051/mateconf/202235503023>.
- [9] L. Xi and S. Geng, "Improved Traffic Sign Detection Algorithm for YOLOv5s," Apr. 2023, doi: <https://doi.org/10.1109/iccea58433.2023.10135461>.
- [10] L. Zhao, Z. Wei, Y. Li, J. Jin, and X. Li, "SEDG-Yolov5: A Lightweight Traffic Sign Detection Model Based on Knowledge Distillation," *Electronics*, vol. 12, no. 2, pp. 305–305, Jan. 2023, doi: <https://doi.org/10.3390/electronics12020305>.
- [11] W. Bai et al., "Two Novel Models for Traffic Sign Detection Based on YOLOv5s," *Axioms*, vol. 12, no. 2, pp. 160–160, Feb. 2023, doi: <https://doi.org/10.3390/axioms12020160>.
- [12] W. Li, Z. Gan, and C. Liu, "A Novel Lightweight Traffic Sign Recognition Model Based on YOLOv5," *Journal of transportation engineering*, vol. 149, no. 5, May 2023, doi: <https://doi.org/10.1061/jtepbs.teeng-7461>.
- [13] S. Zhang, S. Che, Z. Liu, and X. Zhang, "A real-time and lightweight traffic sign detection method based on ghost-YOLO," *Multimedia Tools and Applications*, vol. 82, no. 17, pp. 26063–26087, Jan. 2023, doi: <https://doi.org/10.1007/s11042-023-14342-z>.
- [14] W. Zhao and D. Li, "Traffic sign detection research based on G-GhostNet-YOLOx model," *2022 IEEE 4th International Conference on Civil Aviation Safety and Information Technology (ICCASIT)*, Oct. 2022, doi: <https://doi.org/10.1109/iccasit55263.2022.9986749>.
- [15] W. Song and Shahrel Azmin Suandi, "TSR-YOLO: A Chinese Traffic Sign Recognition

- Algorithm for Intelligent Vehicles in Complex Scenes,” *Sensors*, vol. 23, no. 2, pp. 749–749, Jan. 2023, doi: <https://doi.org/10.3390/s23020749>.
- [16] Aleksej Avramović, D. Sluga, Domen Tabernik, Danijel Skočaj, Vladan Stojnić, and Nejc Ilc, “Neural-Network-Based Traffic Sign Detection and Recognition in High-Definition Images Using Region Focusing and Parallelization,” *IEEE Access*, vol. 8, pp. 189855–189868, Jan. 2020, doi: <https://doi.org/10.1109/access.2020.3031191>.
- [17] Y. Gu and B. Si, “A Novel Lightweight Real-Time Traffic Sign Detection Integration Framework Based on YOLOv4,” *Entropy*, vol. 24, no. 4, pp. 487–487, Mar. 2022, doi: <https://doi.org/10.3390/e24040487>.
- [18] A. Al Khafaji and K. El Abbadi, “Traffic Signs Detection and Recognition Using A combination of YOLO and CNN,” Sep. 2022, doi: <https://doi.org/10.1109/iiccit55816.2022.10010598>.
- [19] S. Liu, J. Zha, J. Sun, Z. Li and G. Wang, "EdgeYOLO: An Edge-Real-Time Object Detector," 2023 42nd Chinese Control Conference (CCC), Tianjin, China, 2023, pp. 7507-7512, doi: 10.23919/CCC58697.2023.10239786
- [20] C.-I. Kim, J. Park, Jong Sun Park, W. Jung, and Y. Lim, “Deep Learning-Based Real-Time Traffic Sign Recognition System for Urban Environments,” *Infrastructures*, vol. 8, no. 2, pp. 20–20, Jan. 2023, doi: <https://doi.org/10.3390/infrastructures8020020>.
- [21] Y. Han and Erdal Oruklu, “Traffic sign recognition based on the NVIDIA Jetson TX1 embedded system using convolutional neural networks,” Aug. 2017, doi: <https://doi.org/10.1109/mwscas.2017.8052891>.
- [22] Tabernik, D., & Skocaj, D. (2020). Deep learning for large-scale traffic-sign detection and recognition. *IEEE Transactions on Intelligent Transportation Systems*, 21(4), 1427–1440. <https://doi.org/10.1109/TITS.2019.2913588>
- [23] Rajesh Kannan Megalingam, Kondareddy Thanigundala, Sreevatsava Reddy Musani, Hemanth Nidamanuru, and L. Gadde, “Indian traffic sign detection and recognition using deep learning,” *International Journal of Transportation Science and Technology*, vol. 12, no. 3, pp. 683–699, Sep. 2023, doi: <https://doi.org/10.1016/j.ijtst.2022.06.002>.
- [24] Emin Güney and Cüneyt Bayılmış, “An Implementation of Traffic Signs and Road Objects Detection Using Faster R-CNN,” *Sakarya university journal of computer and information sciences*, vol. 5, no. 2, pp. 216–224, Aug. 2022, doi: <https://doi.org/10.35377/saucis...1073355>.
- [25] Njayou Youssef, “Traffic sign classification using CNN and detection using faster-RCNN and YOLOV4,” *Heliyon*, vol. 8, no. 12, pp. e11792–e11792, Dec. 2022, doi: <https://doi.org/10.1016/j.heliyon.2022.e11792>.
- [26] F. Wang, Y. Li, Y. Wei, and H. Dong, “Improved Faster RCNN for Traffic Sign Detection,” Sep. 2020, doi: <https://doi.org/10.1109/itsc45102.2020.9294270>.
- [27] J. Mishra and S. Goyal, “An effective automatic traffic sign classification and recognition deep convolutional networks,” *Multimedia Tools and Applications*, vol. 81, no. 13, pp. 18915–18934, Mar. 2022, doi: <https://doi.org/10.1007/s11042-022-12531-w>.
- [28] Soulef Bouaafia, Seifeddine Messaoud, Maraoui Amna, Ahmed Chiheb Ammari, Lazhar Khriji, and Mohsen Machhout, “Deep Pre-trained Models for Computer Vision Applications: Traffic sign recognition,” Mar. 2021, doi: <https://doi.org/10.1109/ssd52085.2021.9429420>.
- [29] X. Zhang and W. Huang, “Traffic sign recognition algorithm based on convolutional neural network,” 2022 3rd International Conference on Computer Vision, Image and

- Deep Learning & International Conference on Computer Engineering and Applications (CVIDL & ICCEA)*, May 2022, doi: <https://doi.org/10.1109/cvidliccea56201.2022.9824113>.
- [30] Nesrine Triki, M. Karray, and M. Ksantini, “A Real-Time Traffic Sign Recognition Method Using a New Attention-Based Deep Convolutional Neural Network for Smart Vehicles,” *Applied sciences*, vol. 13, no. 8, pp. 4793–4793, Apr. 2023, doi: <https://doi.org/10.3390/app13084793>.
- [31] Reem Abdel-Salam, R. Mostafa, and A. H. Abdel-Gawad, “RIECNN: real-time image enhanced CNN for traffic sign recognition,” *Neural Computing and Applications*, vol. 34, no. 8, pp. 6085–6096, Jan. 2022, doi: <https://doi.org/10.1007/s00521-021-06762-5>.
- [32] K. Ren, L. Huang, C. Fan, H. Han, and H. Deng, “Real-time traffic sign detection network using DS-DetNet and lite fusion FPN,” *Journal of Real-time Image Processing*, vol. 18, no. 6, pp. 2181–2191, Apr. 2021, doi: <https://doi.org/10.1007/s11554-021-01102-1>.
- [33] J. Wu and S. Liao, “Traffic Sign Detection Based on SSD Combined with Receptive Field Module and Path Aggregation Network,” *Computational Intelligence and Neuroscience*, vol. 2022, pp. 1–13, May 2022, doi: <https://doi.org/10.1155/2022/4285436>.
- [34] Murat Taşyürek, “ODRP: a new approach for spatial street sign detection from EXIF using deep learning-based object detection, distance estimation, rotation and projection system,” *The Visual Computer*, Mar. 2023, doi: <https://doi.org/10.1007/s00371-023-02827-9>.
- [35] “Geolocating Traffic Signs using Crowd-Sourced Imagery | Proceedings of the 28th International Conference on Advances in Geographic Information Systems,” *ACM Conferences*, 2020. <https://dl.acm.org/doi/10.1145/3397536.3422340>.
- [36] “United States Road Symbol Signs - FHWA MUTCD” *mutcd.fhwa.dot.gov*. <https://mutcd.fhwa.dot.gov/services/publications/fhwaop02084/index.htm#:~:text=A%20white%20background%20indicates%20a>
- [37] “ISO 14819-1:2013” *iso.org*, 2023. <https://www.iso.org/obp/ui/es/#iso:std:iso:14819:-1:ed-2:v1:en>
- [38] “ISO/PAS 21448:2019” *iso.org*, 2023. <https://www.iso.org/obp/ui/#iso:std:iso:pas:21448:ed-1:v1:en>
- [39] “ISO 28564-3:2019” *iso.org*, 2023. <https://www.iso.org/obp/ui/#iso:std:iso:28564:-3:ed-1:v1:en>
- [40] “SIGNAGE STANDARD AND PRACTITIONER’S USER GUIDE” *Programme Design*, vol. 5, TR-506-4. Government of Abu Dhabi. <https://jawdah.qcc.abudhabi.ae/en/Registration/QCCServices/Services/STD/ISGL/ISGL-LIST/TR-506-4.pdf>
- [41] N. Aswini, S. V. Uma, and V. Akhilesh, “Drone to obstacle distance estimation using Yolo V3 network and Mathematical Principles,” *Journal of Physics: Conference Series*, vol. 2161, no. 1, p. 012022, Jan. 2022. doi:10.1088/1742-6596/2161/1/012022
- [42] D. Wilson *et al.*, “Object tracking and geo-localization from street images,” *Remote Sensing*, vol. 14, no. 11, p. 2575, May 2022. doi:10.3390/rs14112575